

数据结构与算法

—北京大学

2026-9-9

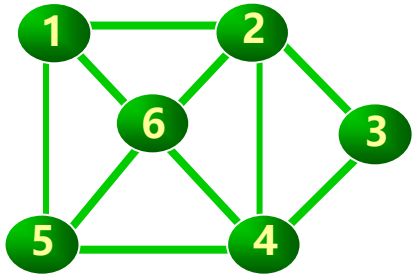
课程基本信息

- **主教材：《数据结构与算法》，张铭、王腾蛟、赵海燕，高等教育出版社，2008年 6月。**
- **慕课平台https://www.icourse163.org/course/PKU-1002534001**
- **算法练习平台 http://dsa.openjudge.cn/**
- **北大课程网 http://course.pku.edu.cn**
- **国家精品课 http://jpk.pku.edu.cn/course/sjjg/**



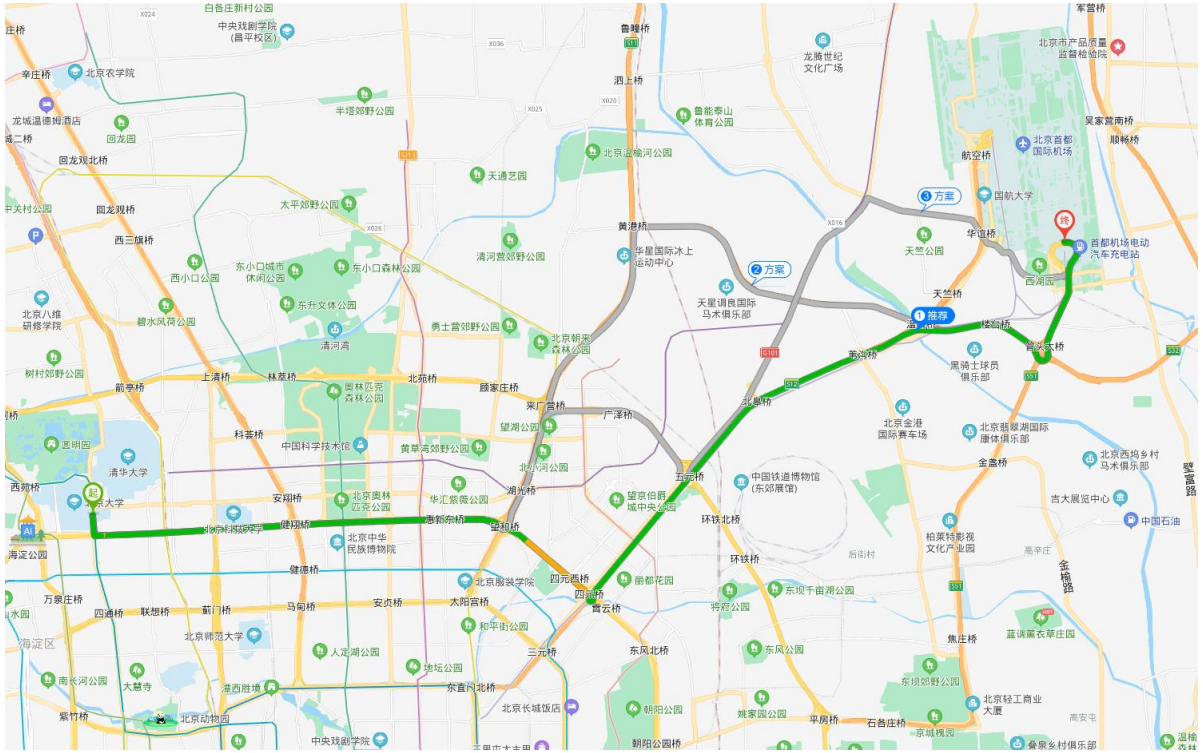
课程理念及目标

- **课程定位：**数据结构和算法是**程序的灵魂**，是计算机学科的基石。“**数据结构与算法**”是计算机专业最重要的**核心课程**
- **课程理念：****问题求解为导向，**培养学生的**创新能力**
- **课程目标：**
  - **问题**抽象、**数据**抽象、**算法**抽象
  - 基于**MOOC资源**的**翻转课堂教学**实践
  - 有效地**组织数据**、设计**高效的算法**、完成**高质量的程序**，从而**解决实际应用的问题**

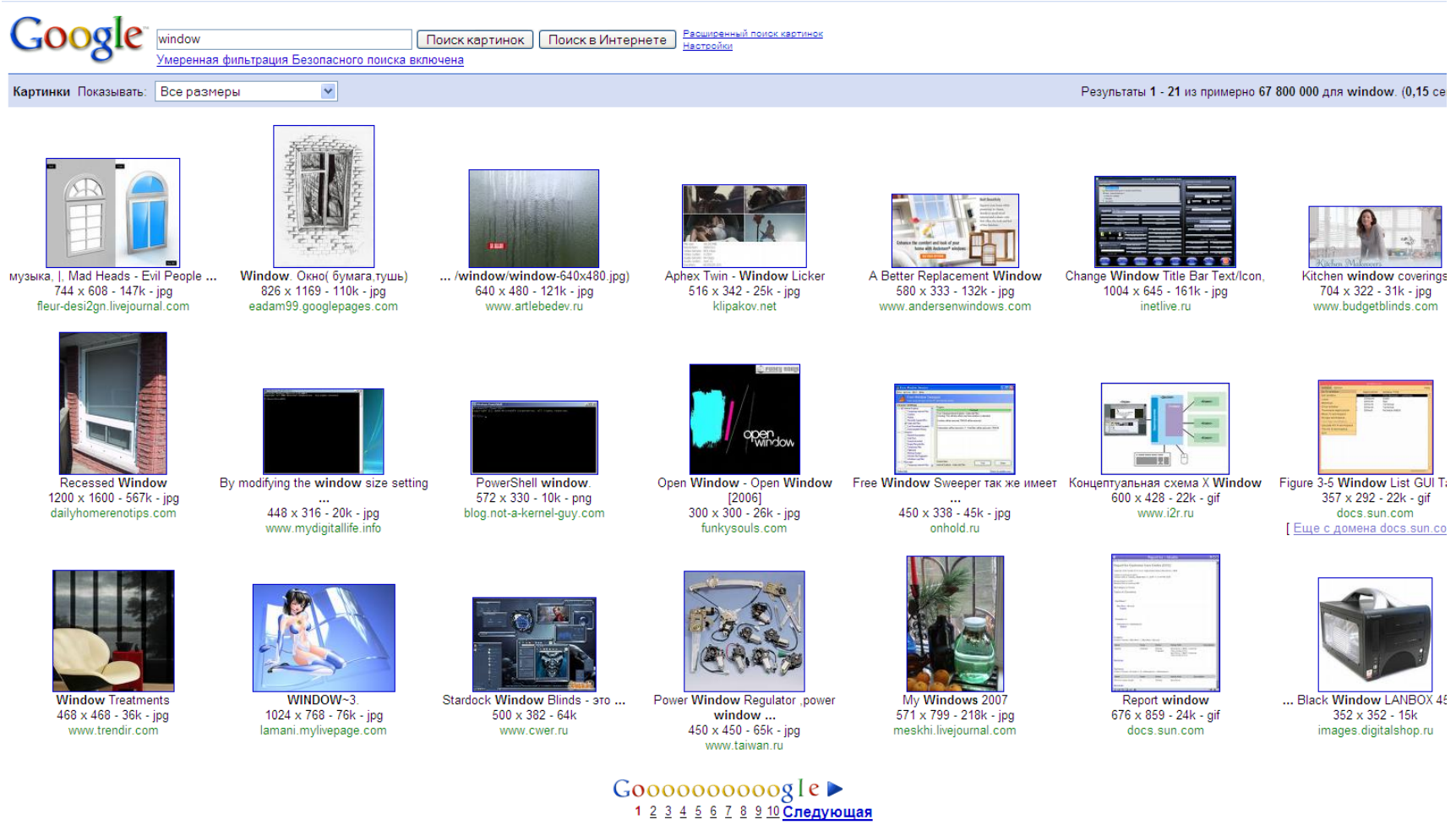


数据结构与算法的应用例子

- 导航路径的计算

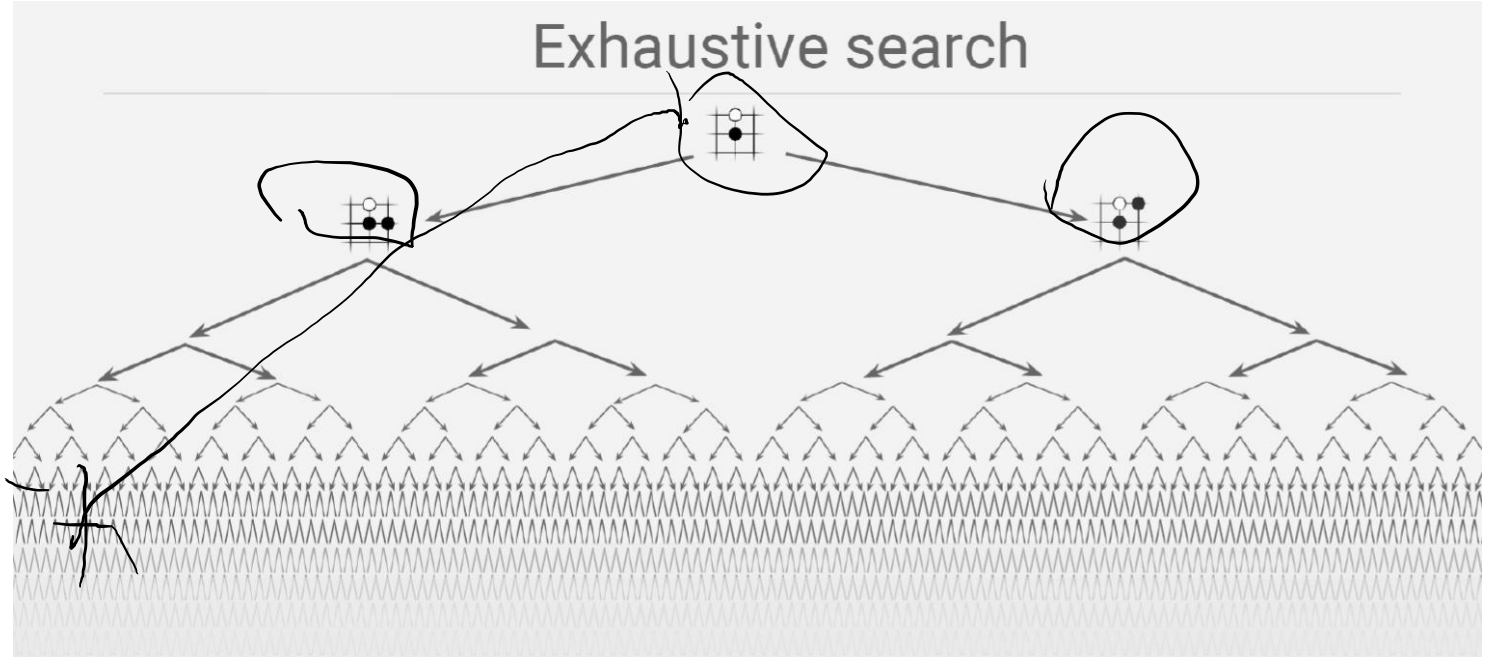


搜索引擎

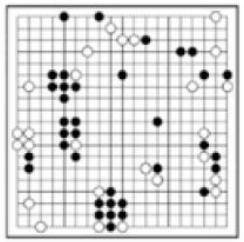

Google
window
Поиск картинок
Поиск в Интернете
Расширенный поиск картинок
Настройки
Умеренная фильтрация Безопасного поиска включена
Картинки Показывать: Все размеры
Результаты 1 - 21 из примерно 67 800 000 для window. (0,15 се
музыка, |. Mad Heads - Evil People ...
744 x 608 - 147k - jpg
fleur-desi2gn.livejournal.com
Window. Окно(бумага,тушь)
826 x 1169 - 110k - jpg
eadam99.googlepages.com
... /window/window-640x480.jpg)
640 x 480 - 121k - jpg
www.artlebedev.ru
Aphex Twin - Window Licker
516 x 342 - 25k - jpg
klipakov.net
A Better Replacement Window
580 x 333 - 132k - jpg
www.andersenwindows.com
Change Window Title Bar Text/Icon,
1004 x 645 - 161k - jpg
inetlive.ru
Kitchen window coverings
704 x 322 - 31k - jpg
www.budgetblinds.com
Recessed Window
1200 x 1600 - 567k - jpg
dailyhomerenotips.com
By modifying the window size setting ...
448 x 316 - 20k - jpg
www.mydigitallife.info
PowerShell window.
572 x 330 - 10k - png
blog.not-a-kernel-guy.com
Open Window - Open Window [2006]
300 x 300 - 26k - jpg
funkysouls.com
Free Window Sweeper так же имеет ...
450 x 338 - 45k - jpg
onhold.ru
Концептуальная схема X Window
600 x 428 - 22k - gif
www.i2r.ru
Figure 3-5 Window List GUI T
357 x 292 - 22k - gif
docs.sun.com
[Еще с домена docs.sun.co
Window Treatments
468 x 468 - 36k - jpg
www.trendir.com
WINDOW~3.
1024 x 768 - 76k - jpg
lamani.mylivepage.com
Stardock Window Blinds - это ...
500 x 382 - 64k
www.cwer.ru
Power Window Regulator ,power window ...
450 x 450 - 65k - jpg
www.taiwan.ru
My Windows 2007
571 x 799 - 218k - jpg
meskhi.livejournal.com
Report window
676 x 859 - 24k - gif
docs.sun.com
... Black Window LANBOX 4
352 x 352 - 15k
images.digitalshop.ru
Goooooooooogle
1 2 3 4 5 6 7 8 9 10 Следующая

电脑游戏

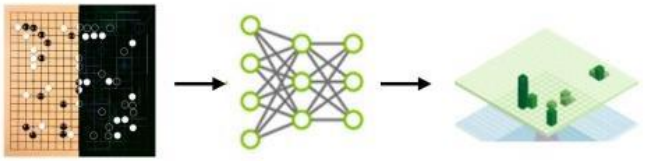




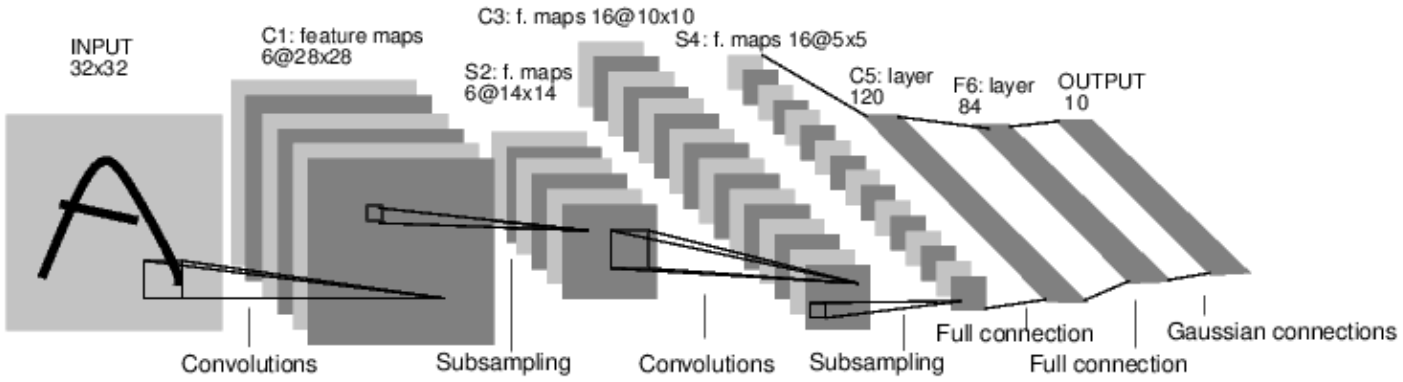
- 几乎无尽的状态空间
- MCTS树搜索算法无效
- 必须缩小探索空间
- 类似人类的下棋方法

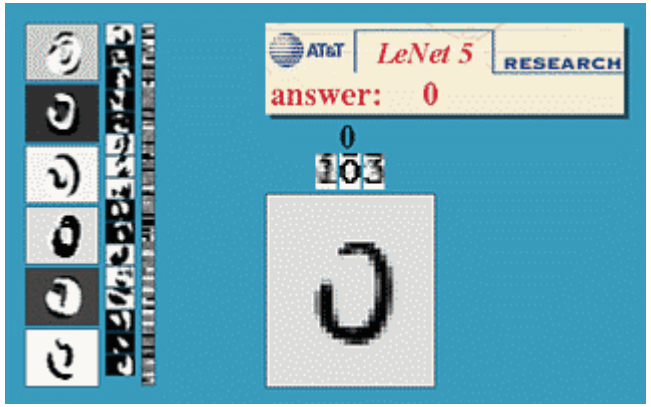


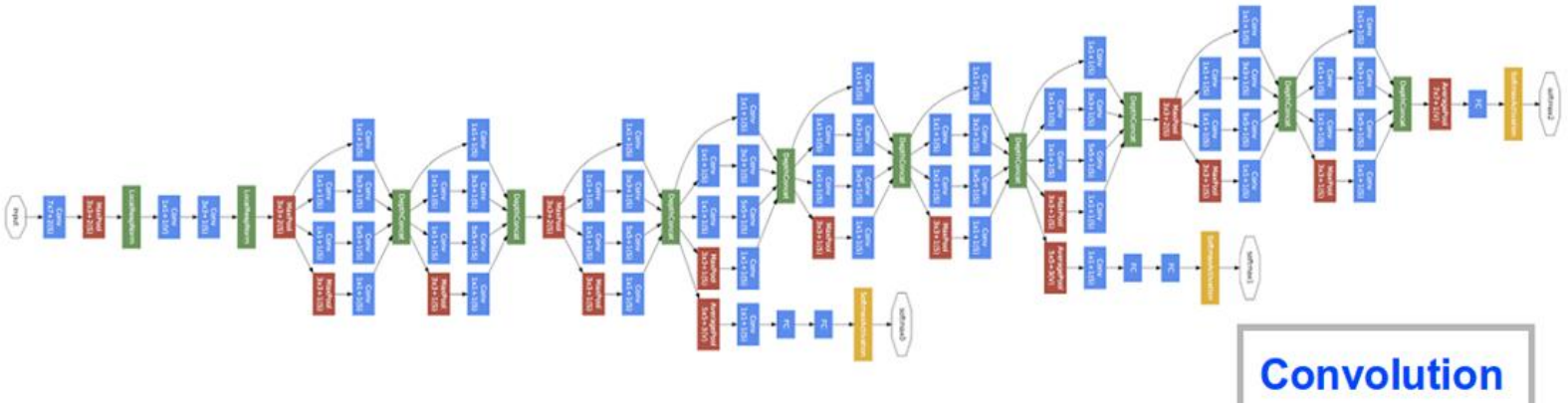
End to End



人工智能


INPUT
32x32
C1: feature maps
6@28x28
S2: f. maps
6@14x14
C3: f. maps 16@10x10
S4: f. maps 16@5x5
C5: layer
120
F6: layer
84
OUTPUT
10
Convolutions
Subsampling
Convolutions
Subsampling
Full connection
Full connection
Gaussian connections


AT&T
LeNet 5
RESEARCH
answer: 0


Convolution


signboard
air_conditioner
truck
car_(automobile)
taillight
license_plate
street_sign
umbrella
trash_can
wheel

人工智能



人工智能

原始视频

TeaCache 需多样本拟合多项式PSNR: 22.8,
2.3x加速

NeurIPS 2025 即插即用
PSNR: 26.7, 2.8x 加速







背后的数据结构与算法

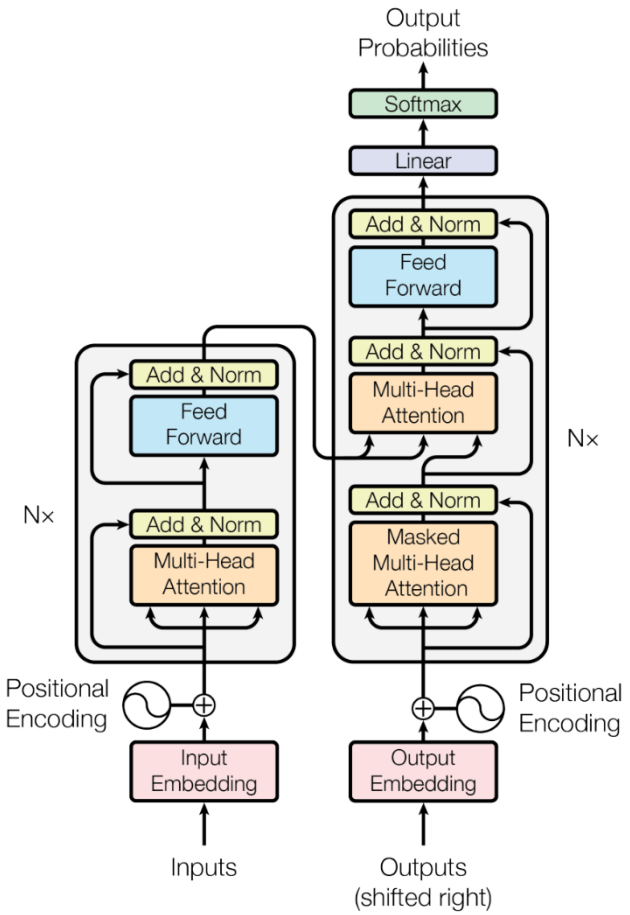
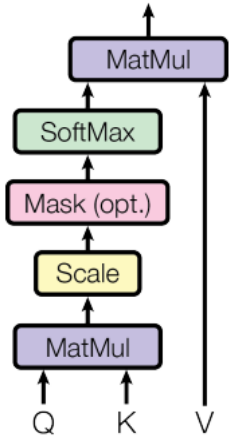


Figure 1: The Transformer - model architecture.



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Ashish Vaswani, et al., Attention is all you need, NeurIPS 2017. 268152 Citations

课程内容

第1章 **概论**

（概念、逻辑结构、存储结构，ADT，算法特征，**算法量度**）

第2章 **线性表**

（线性表ADT和存储结构）

第3章 **栈和队列**

（栈的存储和应用，栈和表达式，**栈和递归，** 队列的概念、循环队列）

第4章 **字符串**

（字符串概念、ADT、**模式匹配**、KMP算法）

第5章 **二叉树**

（二叉树的概念和 ADT，二叉树的深度和宽度遍历，**二叉树的非递归后序周游**，二叉树实现，二叉搜索树、堆，Huffman 树）

第6章 **树**

（树的基本概念，树的深度和宽度遍历，**树的顺序存储**）

课程内容

第7章 **图**

（图的概念，图的遍历和存储，图的拓扑排序，**图的单源最短路径 Dijkstra 算法**、**Floyd 算法**，**最小支撑树**Prim 和 Kruskal 算法）

第8章 **内排序**

（插入排序、冒泡排序和选择排序等简单排序，shell排序，**快速排序**，归并排序，**堆排序** 、**桶排序**，**基数排序**、**地址排序**）

第9章 **文件管理和外排序**

（文件基本概念、置换选择排序、选择树、败方树，多路归并）

第10章 **检索**

（检索的基本概念，顺序检索，**集合检索**，**散列函数**，开散列法，**闭散列，探测算法**）

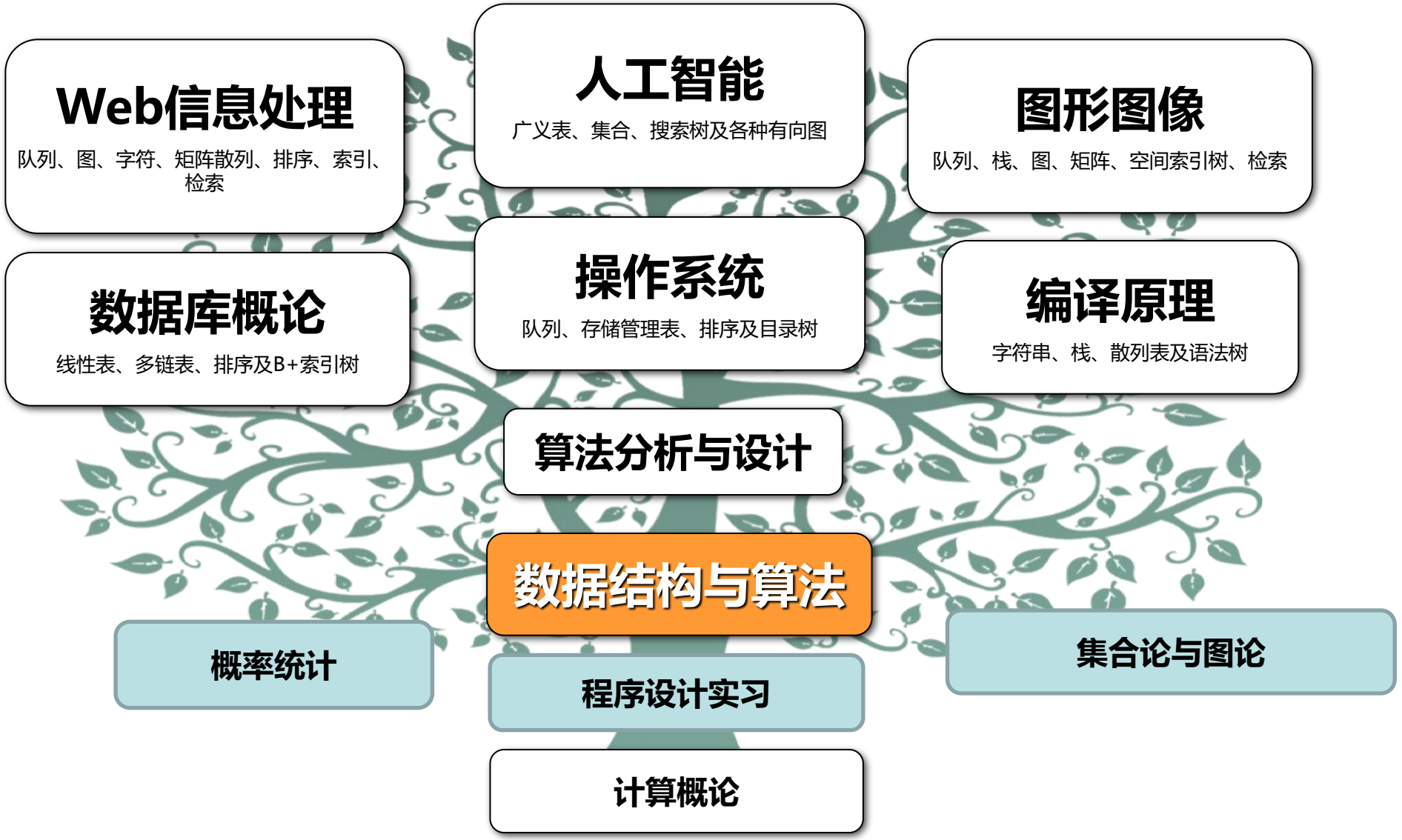
第11章 **索引**

（索引基本概念，线性索引，**倒排索引**，**B/B+树，红黑树**）

第12章 高级数据结构（**了解**，不要求掌握）

（多维数组，矩阵，广义表，**Trie树**，最佳二叉搜索树，**AVL树，自组织伸展树**）

课程地位


Web信息处理
队列、图、字符、矩阵散列、排序、索引、检索
人工智能
广义表、集合、搜索树及各种有向图
图形图像
队列、栈、图、矩阵、空间索引树、检索
数据库概论
线性表、多链表、排序及B+索引树
操作系统
队列、存储管理表、排序及目录树
编译原理
字符串、栈、散列表及语法树
算法分析与设计
数据结构与算法
概率统计
程序设计实习
集合论与图论
计算概论

课程特点

- 基础性
- 理论性
- 实践性
- 挑战性

成绩评定

- **40: 60 (15: 10: 15; 20: 25: 15)**
  - ✓ 平时成绩 40%
    - 作业 15 %： 书面作业 (分析证明为主) 、 考勤 + 课堂参与 + 课堂小测验
    - POJ 编程 10% （ http://dsa.openjudge.cn/ ）
    - MOOC 15%: 基本概念（填空选择、quiz小测、 期中期末考试 (具体参照 https://www.icourse163.org/course/PKU-1002534001 要求)
  - ✓ 闭卷考试 60%
    - 期中 20%
    - 期末 25%
    - POJ机考 15%

教学团队可能进行微调

上机成绩折算

$$总评 = \begin{cases} 0.045 * x, & if\ x \leq 200 \\ 9 + 0.02 * (200 - x), & if\ x > 200 \end{cases}$$

其中x为上机分数

	上机成绩	总评
	50	2.25
	100	4.5
	150	6.75
	200	9
	250	10
	300	11
	350	12
	400	13
	450	14
	500（满分）	15（满分）

教学目标

- **“数据结构 + 算法 = 程序”**
  - 基本的抽象数据类型（ADT） 及其应用
  - 合理组织数据，有效表示数据，有效处理数据
  - 算法的设计分析技术
- **抽象能力**
  - **问题 —— 数据 —— 算法**
- 提高**程序设计的质量**

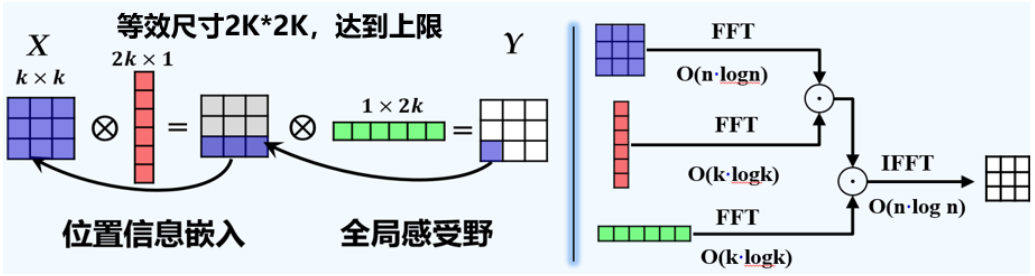
数据结构与算法参考网站

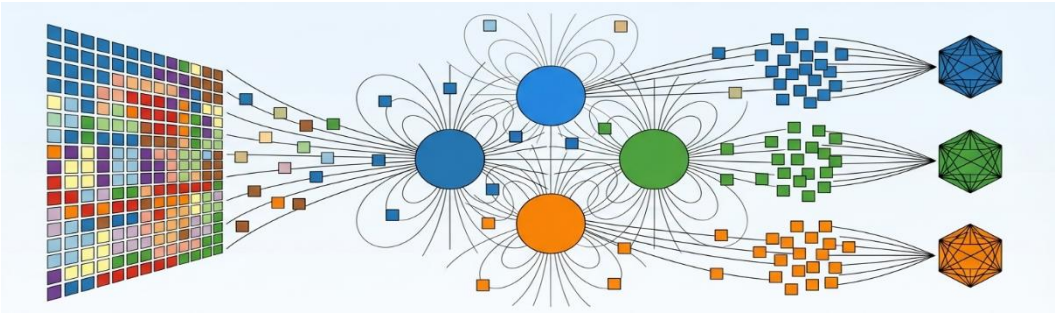
- **Berkeley《数据结构》**
  http://www.cs.berkeley.edu/~jrs/61b/
- **MIT 的《算法导论》 (有OCW链接)**
  http://stellar.mit.edu/S/course/6/sp13/6.006/
- Stanford 算法
  https://www.coursera.org/course/algo
- Princeton 算法课
  https://www.coursera.org/course/algs4partI
- Web 上的术语资源
  http://www.nist.gov/dads/
- **Algorithms in the Real World （后缀树，加密算法，整数规划，生物信息学等)**
  - http://www.cs.cmu.edu/~guyb/realworld.html
- **Advanced Data Structures and Algorithms （自适应数据结构、树堆、随机算法、流数据处理、关联规则)**
  - http://theory.stanford.edu/~rajeev/cs361.html
- **Advanced Data Structures （后缀树/后缀数组，自适应数据结构，高级Trie等)**
  - http://www.cs.biu.ac.il/~moshe/ds1.html

授课教师

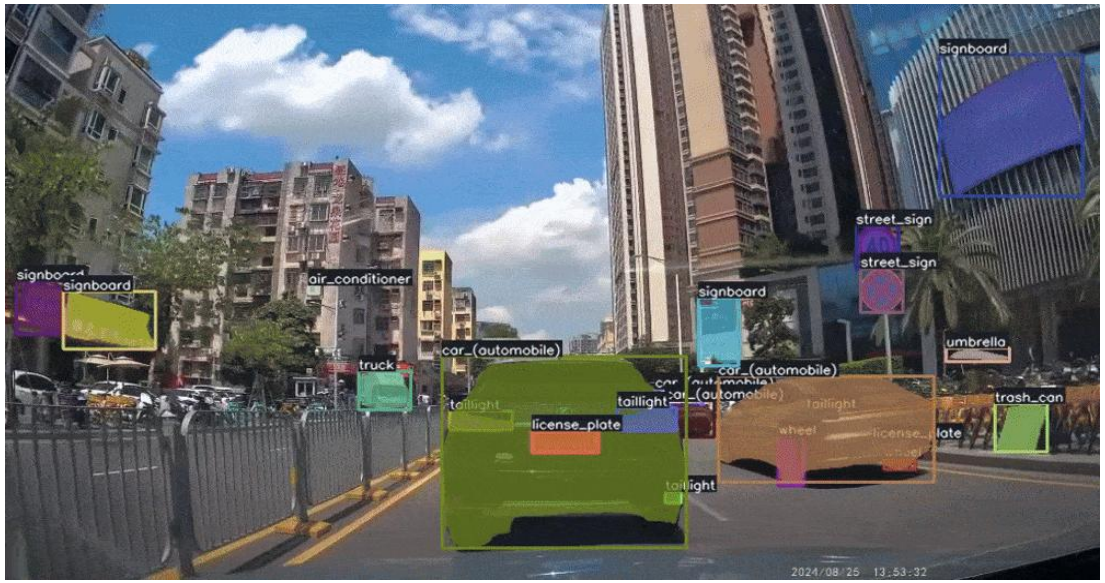
- 主讲教师
  - 张史梁，长聘副教授，slzhang.jdl@pku.edu.cn
  - 计算机学院，视频与视觉技术研究所，多媒体信息处理全国重点实验室
- 研究方向：机器视觉、多媒体内容分析、人工智能

高效、稀疏的网络架构





视觉内容分析与生成



助教与教学安排

- **李少煊**
  - **博士生**
  - **Email: shaoxuanli@stu.pku.edu.cn**
- **宣苏航**
  - **博士生**
  - **Email：xsh1217@stu.pku.edu.cn**
- **郑智馨**
  - **博士生**
  - **Email: zhengzhixin26@stu.pku.edu.cn**

上课时间： 1-16周 每周 周三 3-4节
1-16周 每周 周五 1-2节

上课地点： 理教408
理教408

考试时间： 第1周 周三(2026.12.30)上午

考试类别： 闭卷



群聊: DSA 2026 张史梁老师班

该二维码7天内(9月15日前)有效，重新进入将更新

课程网站





- **作业发布和提交**
- **课件**
- **通知**
- **资料**
- **论坛**
- **。 。 。**

Welcome and Enjoy



数据结构与算法

第一章 概论

张史梁

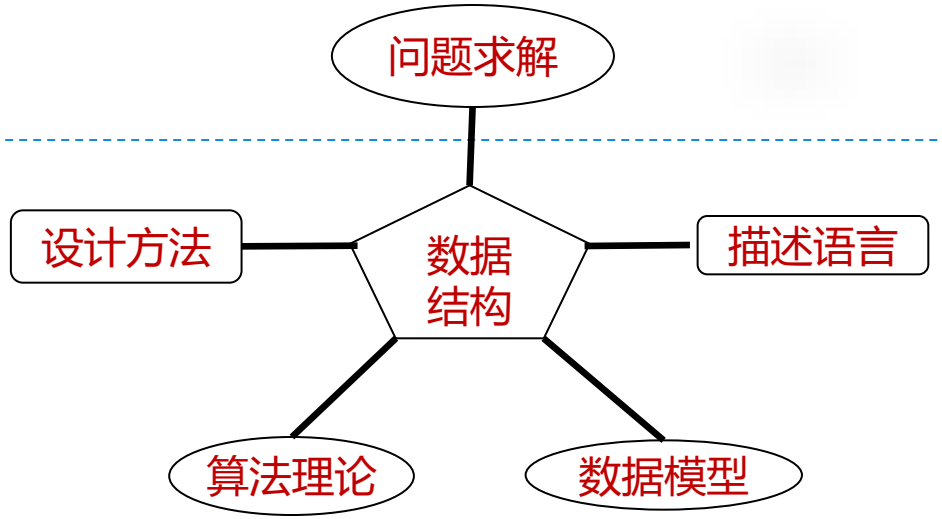
2026-9



第1章　概论

- **问题求解**
- 数据结构及抽象数据类型
- 算法的特性及分类
- 算法的效率度量

1.1 问题求解



- 编写计算机程序的目的?
  - 解决实际的应用问题
- 问题抽象
  - 分析和抽象任务需求，建立问题模型
- 数据抽象
  - 确定恰当的数据结构表示数学模型
- 算法抽象
  - 在数据模型的基础上设计合适的算法
- 数据结构 + 算法，进行程序设计
  - 模拟和解决实际问题

多叉路口交通信号灯的管理

问题分析：首先需要分析一下所有车辆的行驶路线的冲突问题。这个问题可以归结为对车辆的可能行驶方向作某种分组，对分组的要求是使任一个组中各个方向行驶的车辆可以同时安全行驶而不发生碰撞。

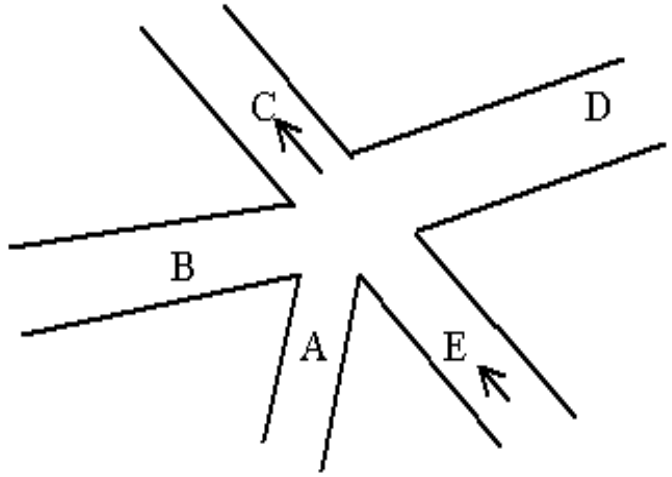


图 1.1　一个交叉路口的模型

(C和E单行，其它双行)

根据这个路口的实际情况可以确定13个可能通行方向：A→B，A→C，A→D，B→A，B→C，B→D，D→A，D→B，D→C，E→A，E→B，E→C，E→D。

为了叙述方便，我们下面把A→B简写成AB，并且用一个小椭圆把它框起来，在不能同时行驶的路线间画一条连线(表示它们互相冲突)，便可以得到下图

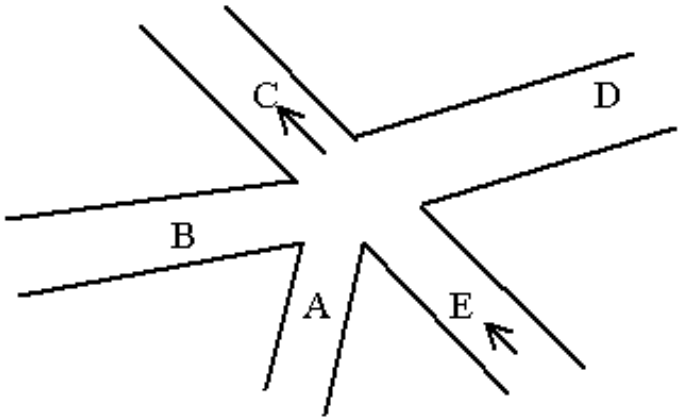


图 1.1　一个交叉路口的模型

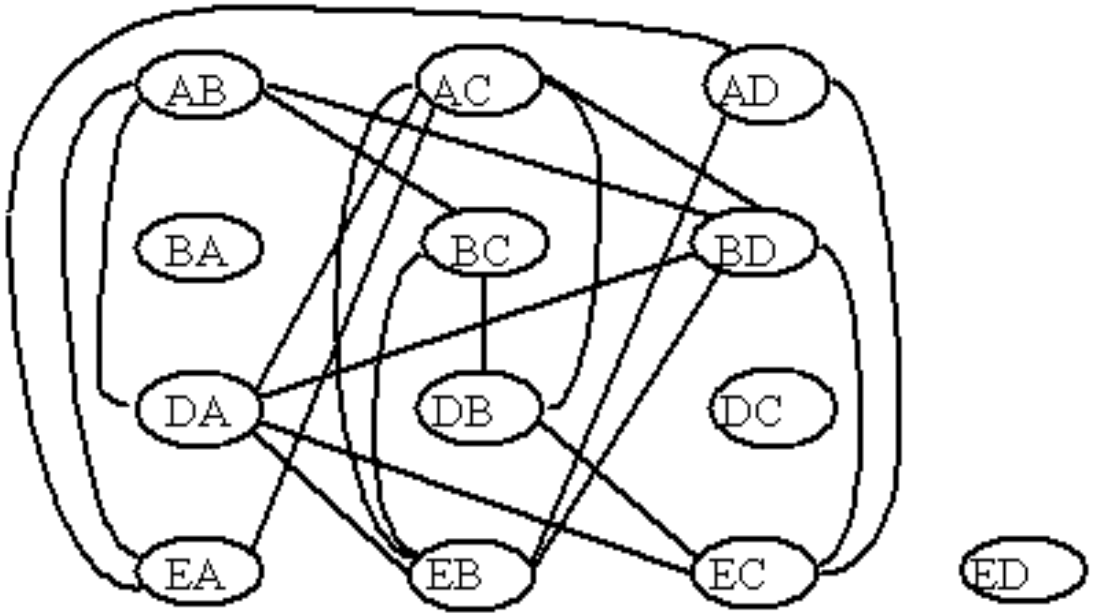


图 1.2　交叉路口的图式模型

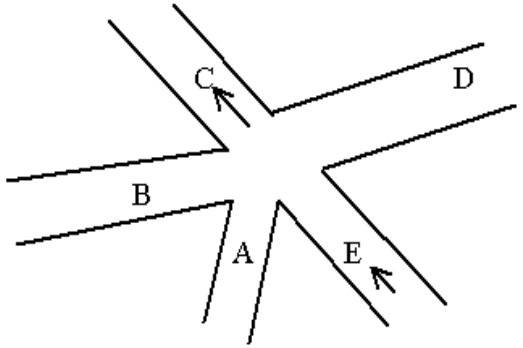


图 1.1　一个交叉路口的模型

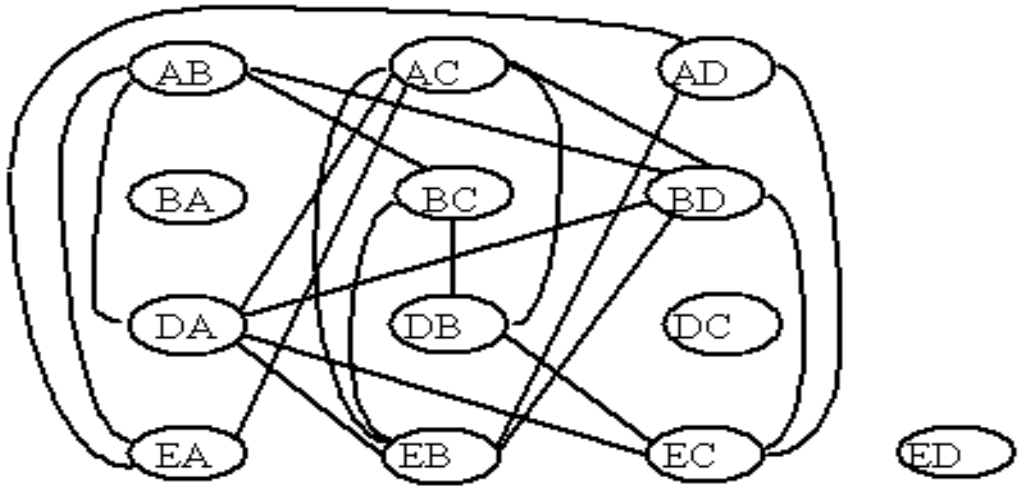


图 1.2　交叉路口的图式模型

这样做就把要解决的问题借助图的模型变成了另一个抽象问题：要求将图1.2中的结点分组，使有线相连(互相冲突)的结点不在同一个组里。

- **可行解：仅满足安全性的普通解。**
- **最优解：分组数最少的解。**
- **次优解：分组数接近最优解的可行解**

着色问题

- 如果把上图中的一个结点理解为一个国家，结点之间的连线看作两国有共同边界，上述问题就变成著名的“着色问题”
- 即求出要几种颜色可将图中所有国家着色，使得任意两个相邻的国家颜色都不相同。



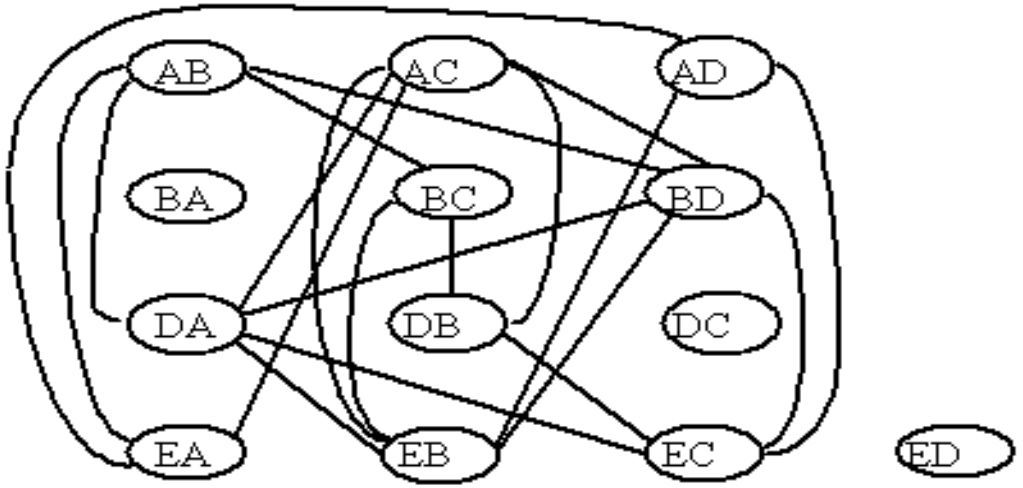


图 1.2　交叉路口的图式模型

着色问题求解：穷举

- 穷举法：
  - 从分为1、2、3...组开始考察，逐个列举出所有可能的分组方案，检查这样的分组方案是否满足要求。
  - 首先满足要求的分组，自然是问题的最优解。
    - 将所有结点放在一个组里是否可行（一种颜色），考察组内结点是否有连线，没有则成功；
    - 否则放在两个组里是否可行，穷举出所有可能的分组方案，以此类推三组、四组...
  - 对结点少的问题(称为“规模小的”问题)还可以用
  - 对规模大的问题，由于求解时间会随着实际问题规模的增长而指数性上升，使计算机无法承受。

着色问题求解：贪心法

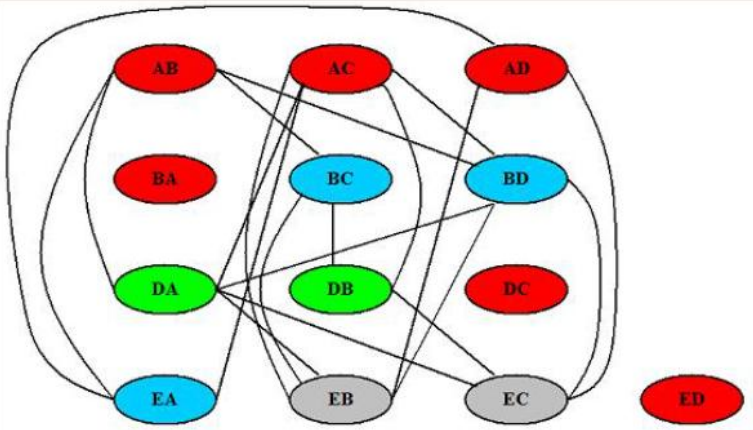
- 1. 用一种新颜色给尽可能多的顶点着色
  - (1) 选择一个未着色的顶点并着该颜色
  - (2) 扫描其他未着色的点，考察其邻接点是否着了该颜色，若没有就着该颜色
- 2. 换一种颜色重复1，
- 直到所有顶点全部着色为止

红色：AB AC AD BA DC ED

蓝色：BC BD EA

绿色：DA DB

灰色：EB EC



着色问题求解

程序实现

- 数据类型选择
  - 使用什么数据类型来表示地图，以及使用什么样的数据类型表示一组国家等。
  - 如：使用一个图结构表示地图；使用国名（图中结点名）的集合表示国家的分组。

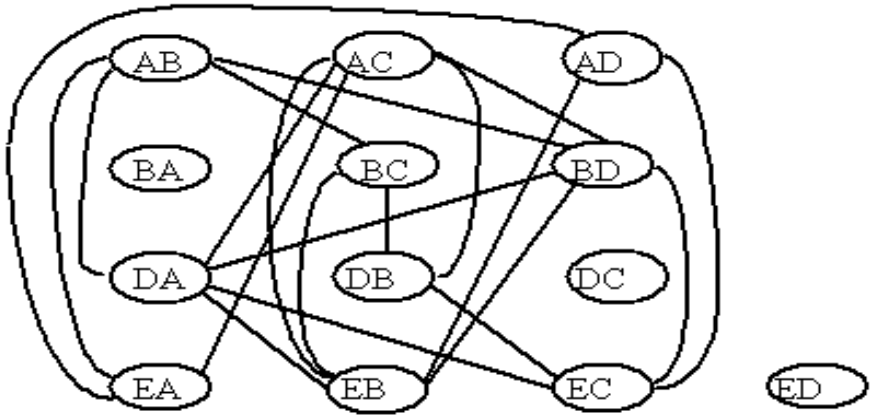


图 1.2　交叉路口的图式模型

程序描述

- 设需要着色的图是G，集合V1包括所有未被着色的结点，着色开始时V1是G所有结点集合。
- NEW表示已经着色的结点集合。
- 贪心法的的伪码描述：

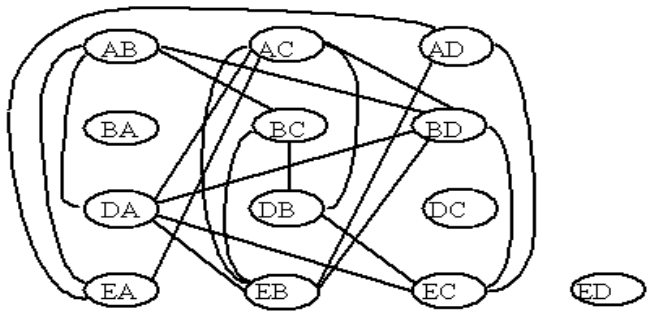


图 1.2　交叉路口的图式模型

```
// 找到一组可以着相同颜色的节点集合
置NEW为空集合;
for 每个v ∈ V1 do
        if  v与NEW中所有结点间都没有边
              从V1中去掉v ;
              将v加入NEW ;
```

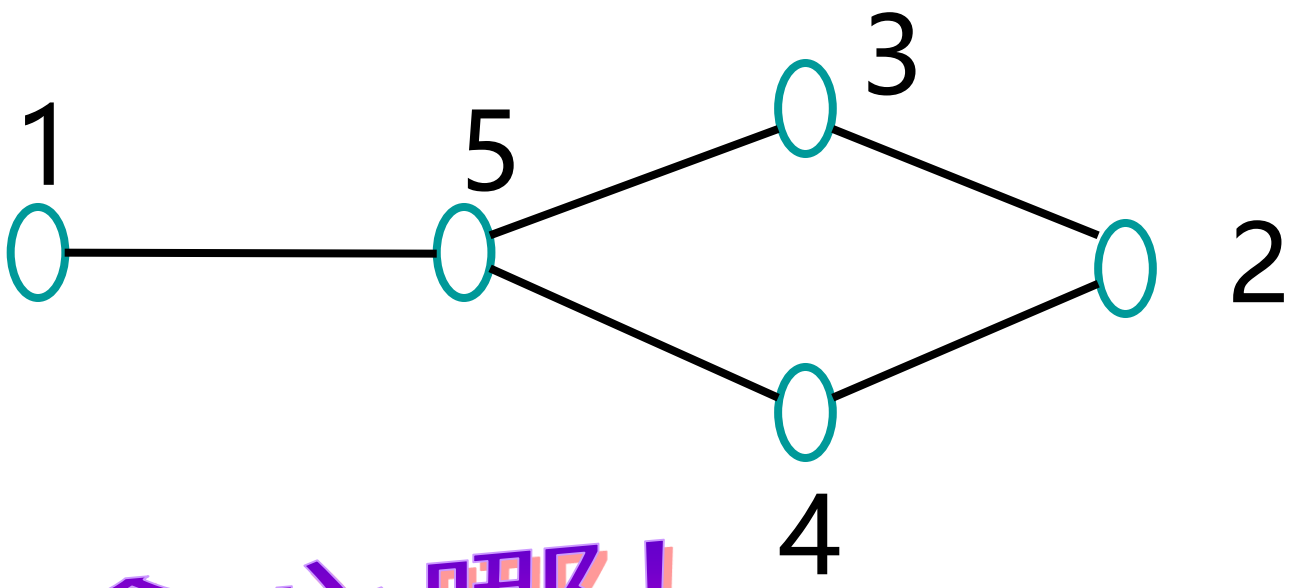
- 着色程序可以反复调用这段伪码，直到V1为空
- 每次调用选择一种新颜色，这段伪码执行的次数就是需要的不同颜色个数。

编程

```
int colorUp(Graph G) //输出所需的颜色数量和每种颜色对应的节点
   {      int color = 0;
          V1 = G.V;
          while(!isSetEmpty(V1)) //如果V1非空
          {          emptySet(NEW);  //清空NEW
                     while (v ∈V1 && notAdjacentWithSet(NEW, v, G))
                     {          addToSet(NEW, v); // v加入NEW
                                removeFromSet(V1, v); //将v从V1中删除
                     }
                     printSet(NEW); // 输出当前NEW中的节点
                     ++color; // 所需颜色数量+1
          }
          return (color);
   }
```

贪心法近似解

- 按1，2，3，4，5顺序着色



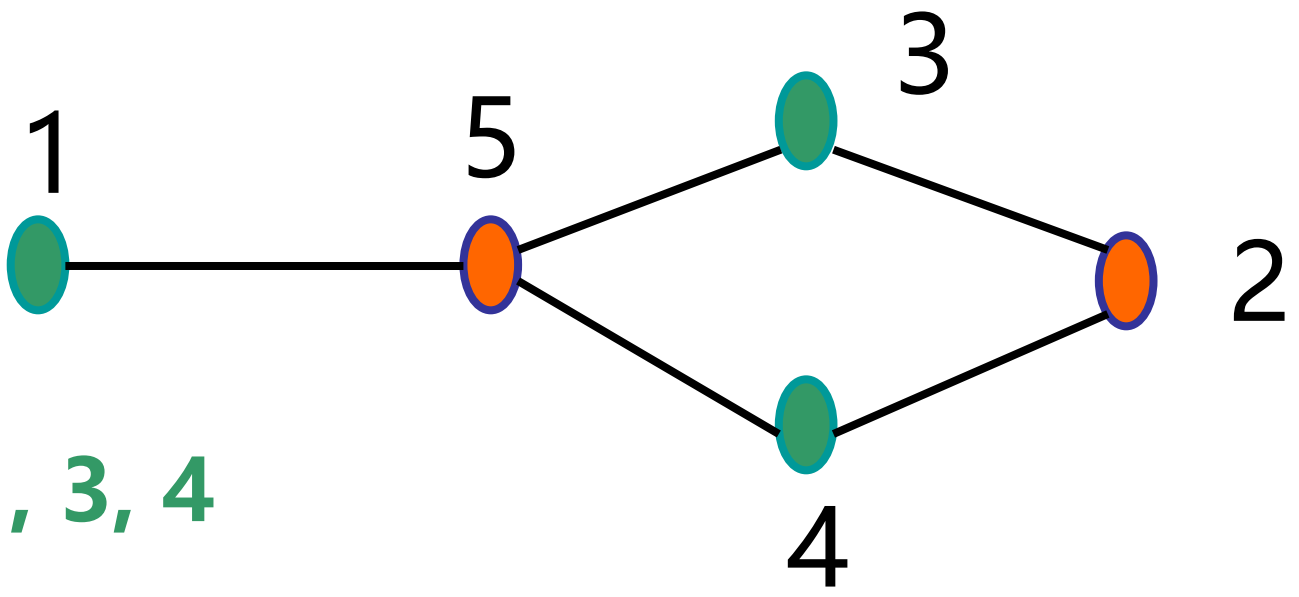
1，2

3，4

5

贪心哪！

最优解



1, 3, 4

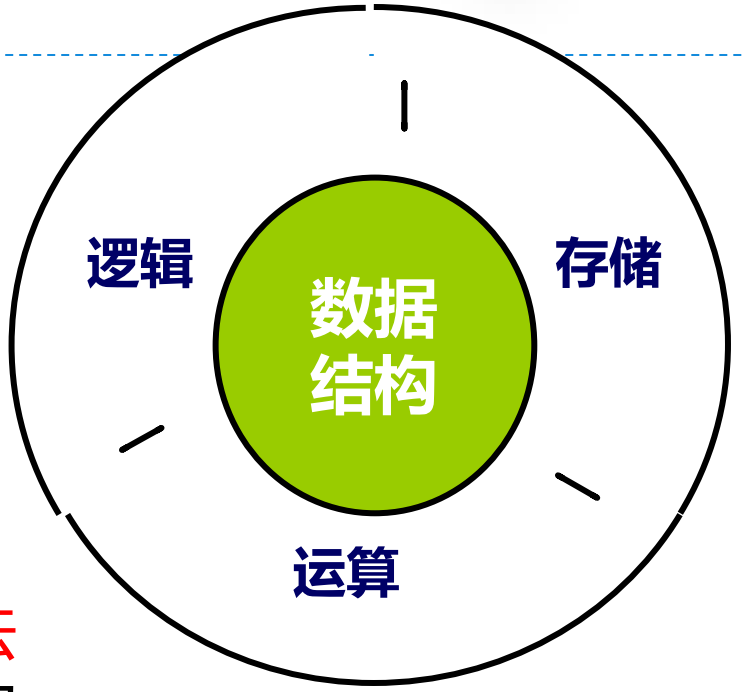
2， 5



第1章　概论

- 问题求解
- **数据结构及抽象数据类型**
- 算法的特性及分类
- 算法的效率度量

1.2 什么是数据结构



数据（Data）：

指能够被计算机识别、存储和加工处理的信息载体。在计算机科学中是所有能输入到计算机中并能被计算机程序处理的符号的总称。

数据结构:

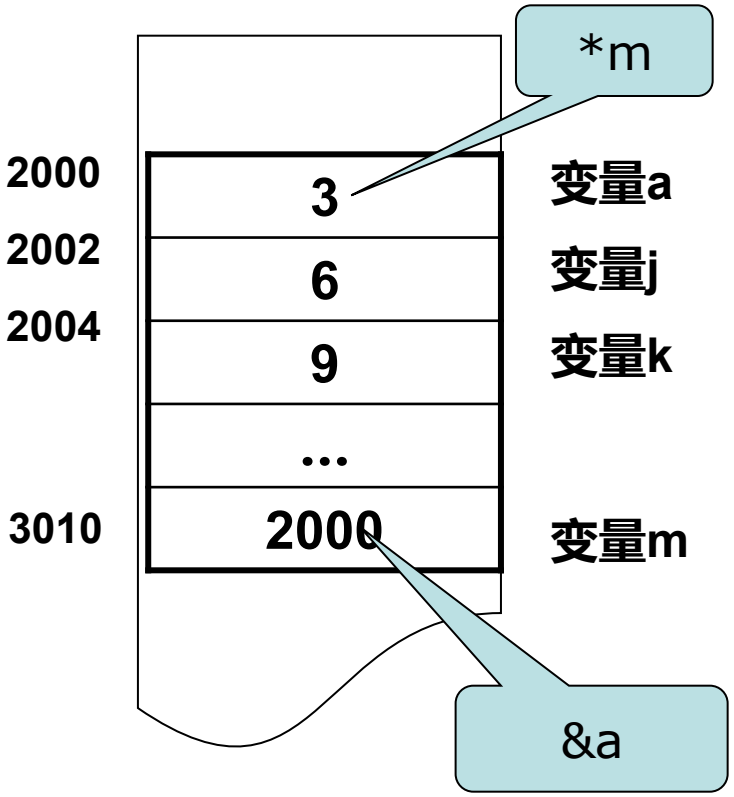
- 按照逻辑关系组织起来的一批数据，按一定的存储方法把它存储在计算机中，在这些数据上定义了一个运算的集合
  - 涉及数据之间的逻辑关系、数据在计算机中的存储表示和在这种结构上一组能执行的操作（运算）

基本数据类型

- 整数类型(integer)
- 实数类型(real)
- 布尔类型(boolean)
  - C++语言中0表示false，非0表示true
  - 也支持 false，true 保留字
- 字符类型(char)
  - ASCII用**单个字节**表示字符
  - 汉字符号需用**2个字节**的编码
  - Unicode，GB，Big5，HZ

指针类型

- 指针类型(pointer)：指向某一内存单元的**地址**
  - 32 bit机器，4个字节表示一个指针
  - 64bit的机器， 8个字节表示一个指针
- 指针操作
  - 分配地址
  - 赋值（另一个指针的地址值，NULL空值）
  - 比较两个指针地址
  - 指针增减一个整数量



复合数据类型

- 基本数据类型/复合类型组成的复杂结构

例如，

- 数组： int A[100];
- 结构： typedef struct {} B;
- 类： class C { };

C++中结构体和类的区别是什么?

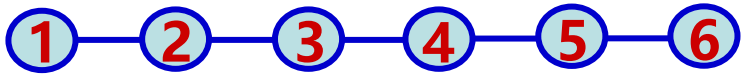
结构的默认访问类型是public，而类的默认访问类型为private
结构是实值类型，而类则是引用类型； 结构使用栈存储，而类使用堆存储
栈的执行效率要比堆的执行效率高，但是栈资源却很有限，不适合处理逻辑复杂的大对象

数据结构三要素

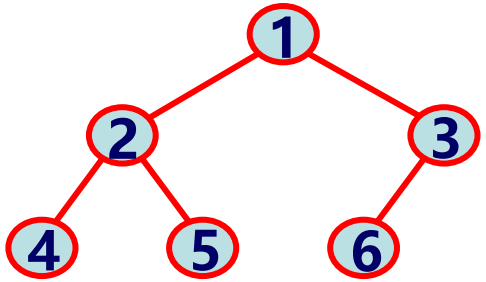
- 逻辑结构:
  - 定义了结构中的基本元素之间的相互关系。
  - 如：图关系，树关系
- 存储结构:
  - 给出了结构中的基本元素及其之间关系的存储方式，包括元素的表示和关系的表示。
  - 如：图、树在内存中存储图
- 数据的运算:
  - 这个结构具有的行为特征，体现为在存储结构上具体实现的算法。
  - 如：判断两个节点是否在图中相连，树上节点间的从属关系

逻辑结构：数据结构的逻辑组织

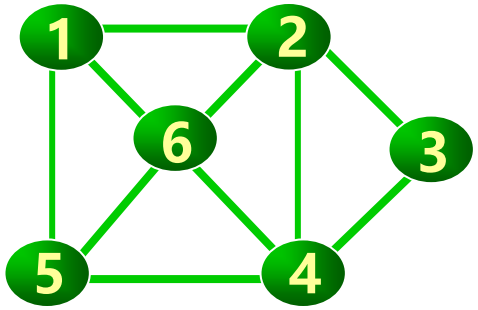
- **线性结构**
  - 线性表（表，栈，队列，串等）



- **非线性结构**
  - 树（二叉树，Huffman树，二叉检索树等）
  - 图（有向图，无向图等）



- **图 ⊇ 树 ⊇ 二叉树 ⊇ 线性表**

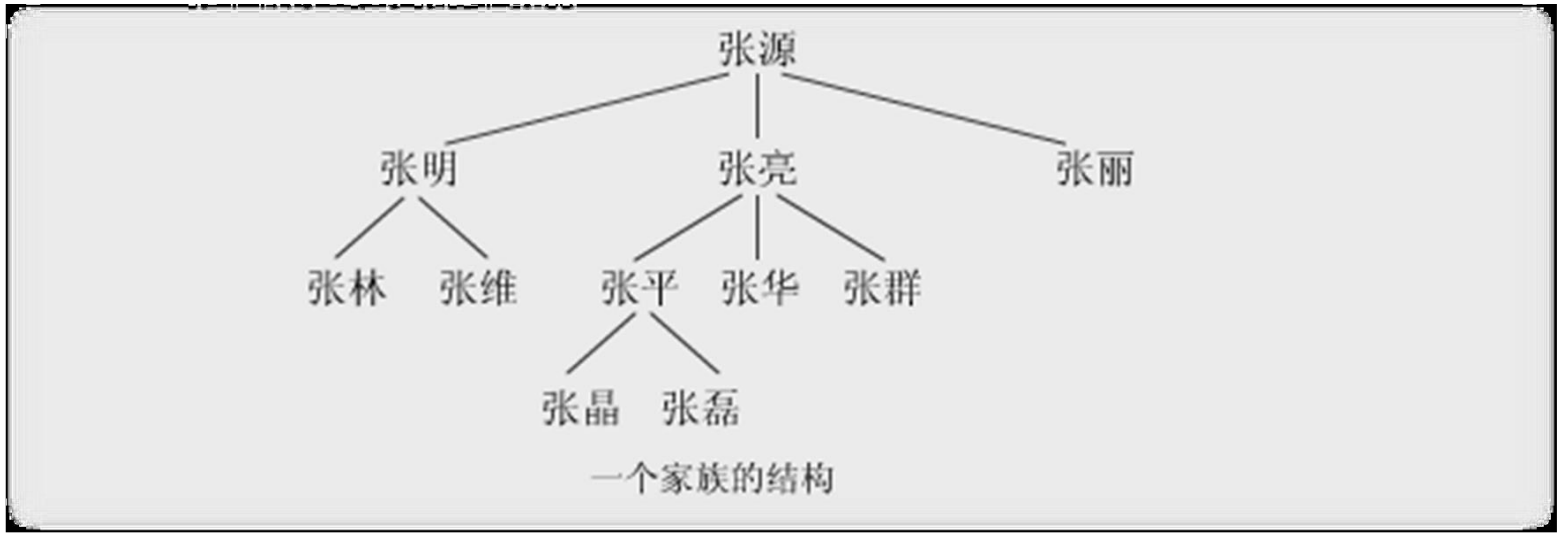


逻辑结构

- 逻辑结构的表示
  - 用一组数据（表示为结点集合**K**），及数据间的二元关系（关系集合**R**）表示：**（K，R）**
    - ✓ **K**由有限个结点组成，代表一组有明确结构的数据集
    - ✓ **R**是定义在集合**K**上的一组关系，每个关系**r**∈**R**都是**K**×**K**上的二元关系，描述结点间的逻辑关系
    - ✓ 若<k1,k2> ∈R, 则称k1为k2的前驱, k2为k1的后继。没有前驱的结点为开始结点, 没有后继的结点为终端结点。

举例

- 家族成员-树状数据结构
  - 每个成员个体作为结点（实体），全部成员组成结点集**K**
  - 家族中各类亲属关系就是一组关系**R**
    - 其中如父子关系**r**、兄弟关系**r***、和妯娌关系**r'**等



一个家族的结构

结点的类型

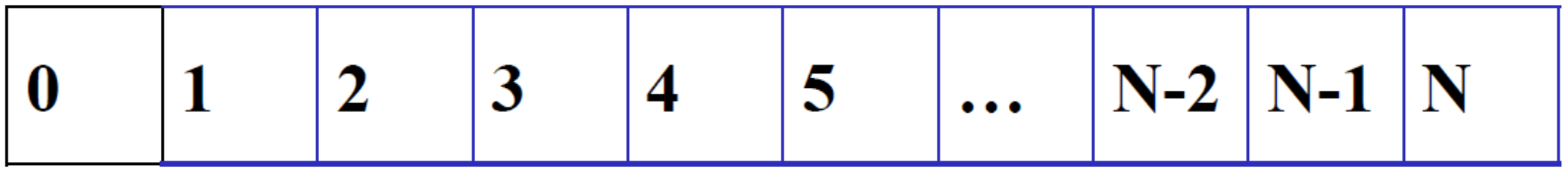
- 基本数据类型
  - 整数类型**(integer)**、实数类型**(real)**、字符类型**(char)**和指针类型**(pointer)**
- 复合数据类型
  - 由基本数据类型组合而成的数据结构类型，如数组、结构类型等
  - 复合数据类型本身，又可参与定义更为复杂的结点类型
- 结点的类型可以根据应用的需要来灵活定义

结构（关系）类型

- 逻辑结构（**K, R**）的分类，讨论重点在关系集**R**上
- 用**R**的性质来刻划数据结构的特点，并进行分类
  - 集合结构（**set structure**），R为空
  - 线性结构（**linear structure**）
  - 树型结构（**tree structure**）
  - 图结构（**graph structure**）

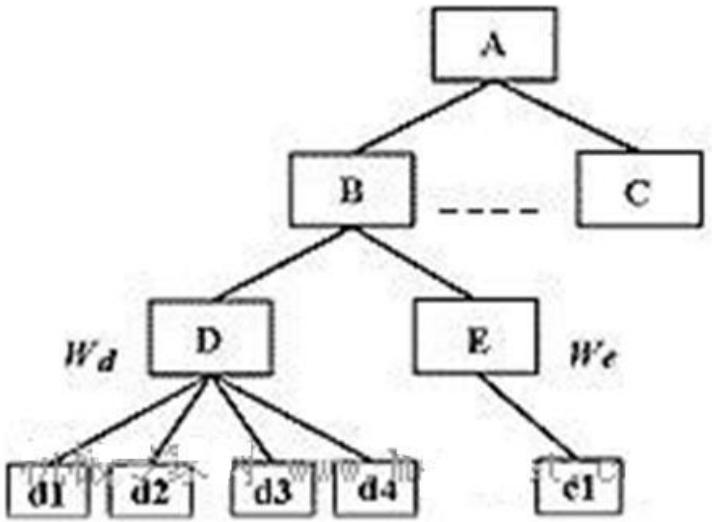
线性结构

- K中每个结点最多只有一个前驱和一个后继
  - 数据元素之间仅存在一对一的关系（唯一后继、唯一前驱）
  - 线性表，栈，队列



树形结构

- 亦称层次结构，每个结点可有多于1个‘后继’，但只有唯一的‘前驱’
- 最高层结点称为根（root）结点，无父结点
- 二叉树、三叉树...



图型结构

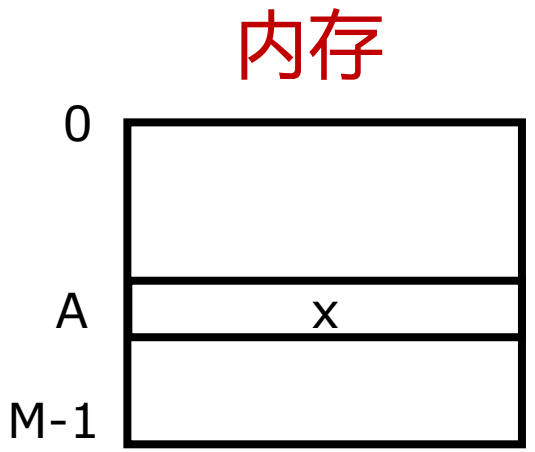
- 图结构亦称网络结构
  - 交通网、因特网、社会网络等
- K中结点的前驱、后继结点的个数都不作限制
- 树型结构和图型结构的区别是
  - 每个结点是否仅仅有一个前驱
- 线性结构和树型结构的区别是
  - 每个结点是否仅仅有一个后继

数据的存储结构

- **逻辑结构到物理存储空间的映射**

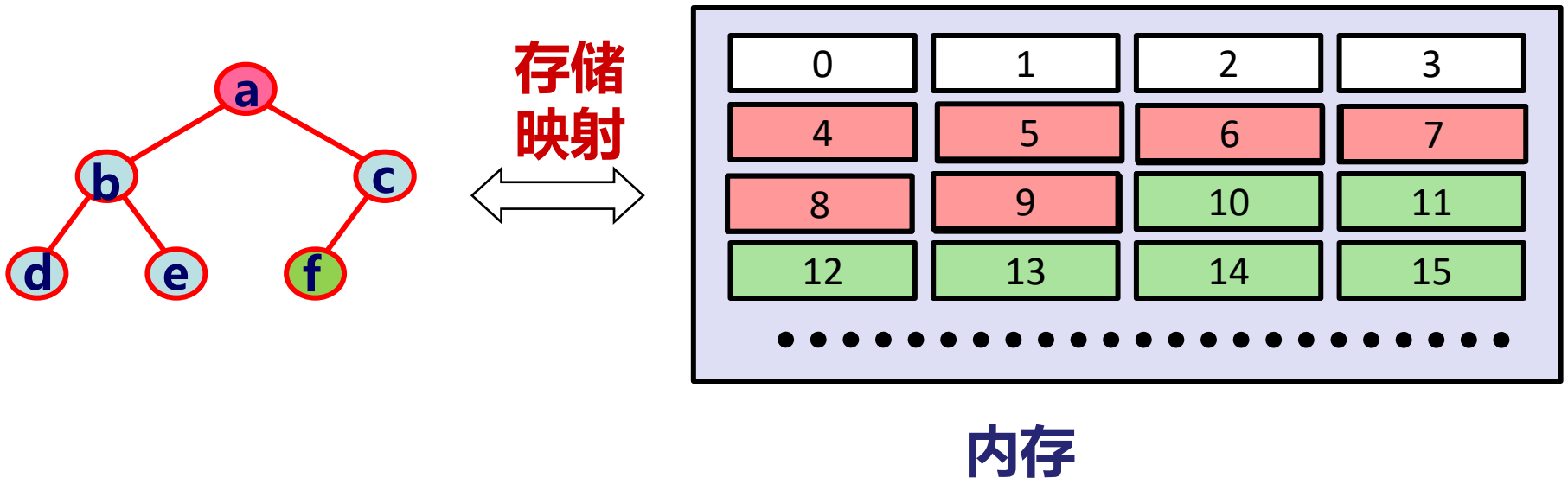
计算机主存储器（内存）

- **非负整数**地址编码，**相邻单元**的集合
  - 基本单位是字节
  - **按地址随机访问**
  - 访问不同地址所需时间基本相同



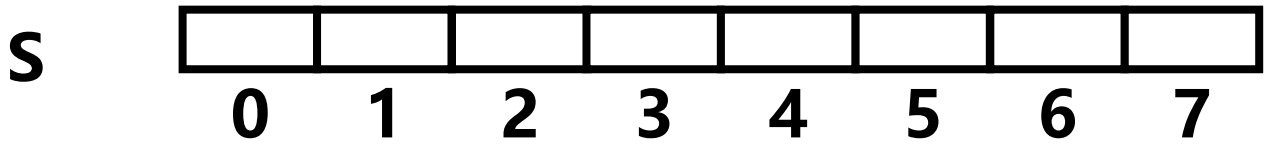
数据的存储结构

- 对逻辑结构（K，r），其中 r∈R
  - 对结点集 K 建立一个从 K 到存储器 M 的单元的映射：K→M，对于每一个结点 j∈K 都对应一个**唯一**的**连续存储**区域c ∈ M

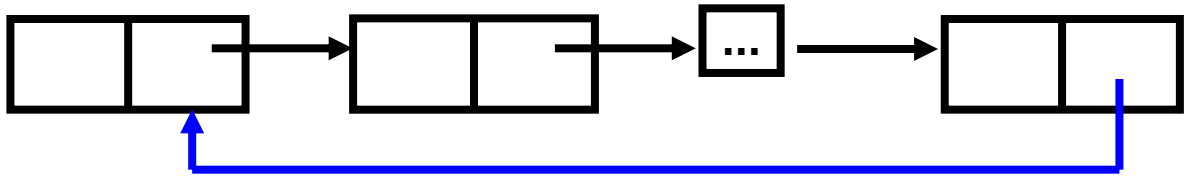


数据的存储结构

- 关系元组 $(j_1, j_2) \in r$
  (其中j_1，$j_2 \in$ K 是结点)
  - 顺序：存储单元的顺序地址



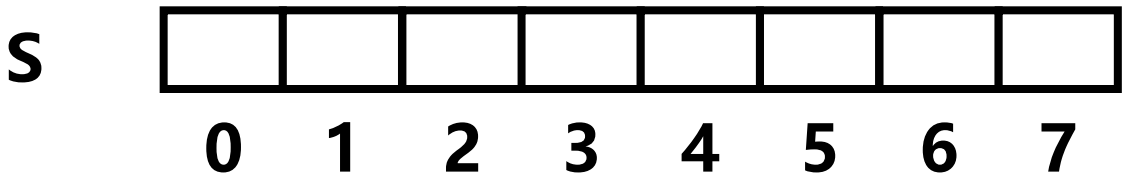
  - 链接：指针的地址指向关系



- 四类：**顺序、链接、索引、散列**

顺序方法

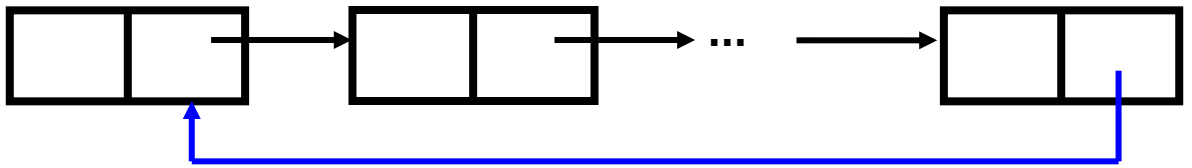
- 把一组结点存储在按**地址相邻的顺序存储**单元里
- **存储单元的顺序**，表达了结点间的逻辑后继关系
- 数组（向量）：**按下标访问**



M_{00}	M_{01}	M_{02}
M_{10}	M_{11}	M_{12}
M_{20}	M_{21}	M_{22}

$$M[i][j]=M[0][0]+(k*i+j)*(\text{元素尺寸})$$

链接方法



- 链接法： 结点的存储结构中附加指针
  - **数据域 E、指针域 P**
  - **指针的指向**，表达了两个结点的逻辑后继关系

- 存储密度($\rho \leq 1$) 数据本身占整个结构存储比率

例如，链表的存储密度

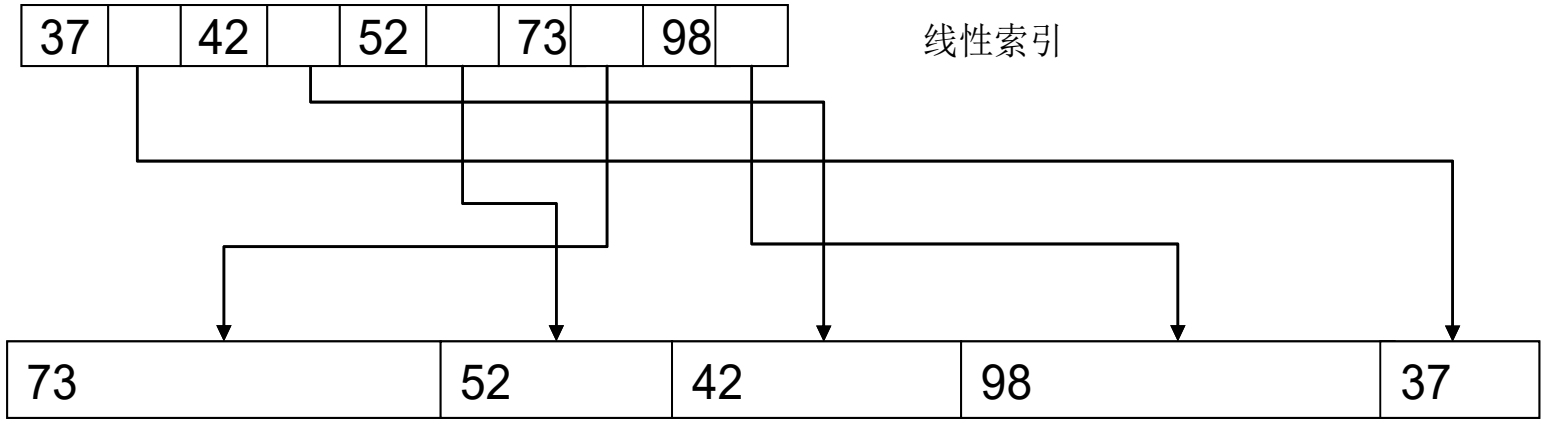
$$\rho = n \times E / n(P+E) = E/(P+E)$$

(n表示线性表中当前元素的数目)

索引方法

- 索引映射函数Y：Z→D
  - 结点的**索引值** z∈Z
  - 结点的**存储地址** d∈D

索引表 S



散列方法

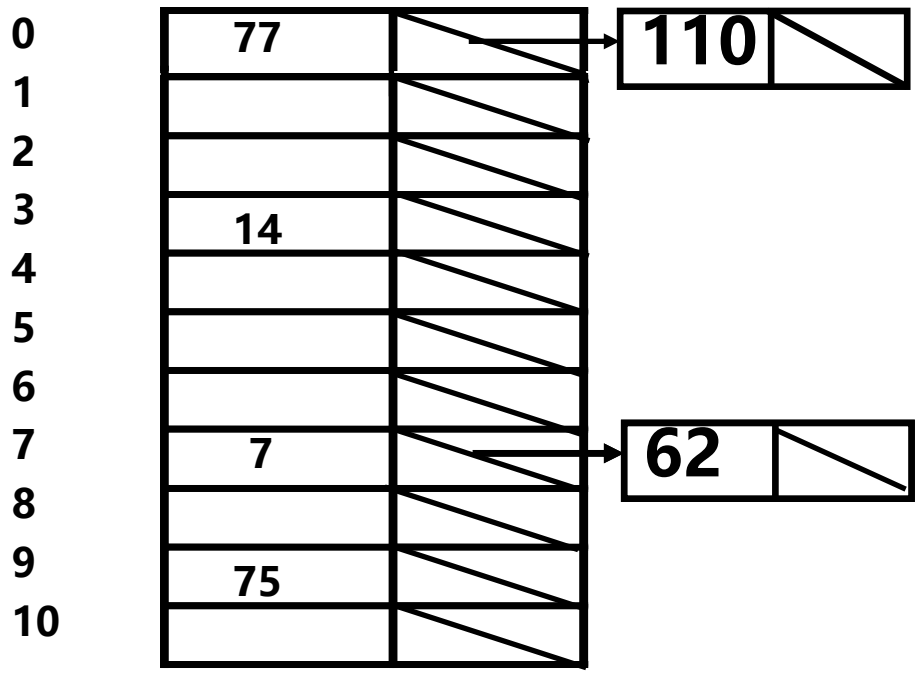
- 散列是索引方法的延伸和扩展

- **散列函数**：将关键码 s **映射** 到非负整数 z

  h: S → Z

  对任意的 s∈ S，散列函数 h(s)=z，z∈ Z

散列（hashing）的方法



$$H(s) = s \% 11$$

数据的运算

- 定义在逻辑结构上的一系列操作以及这些操作在存储结构上的实现；
- 常用的运算：检索、插入、删除、定位、修改、排序等；
- 数据的运算是定义在逻辑结构上的，而具体实现是基于存储结构

抽象数据类型

- 简称**ADT** (Abstract Data Type)
  - 定义了一组运算的数学模型
  - 与物理存储结构无关
  - 使软件系统建立在数据之上(面向对象)

- **模块化**的思想的发展
  - 隐藏运算实现的细节和内部数据结构
  - 软件复用

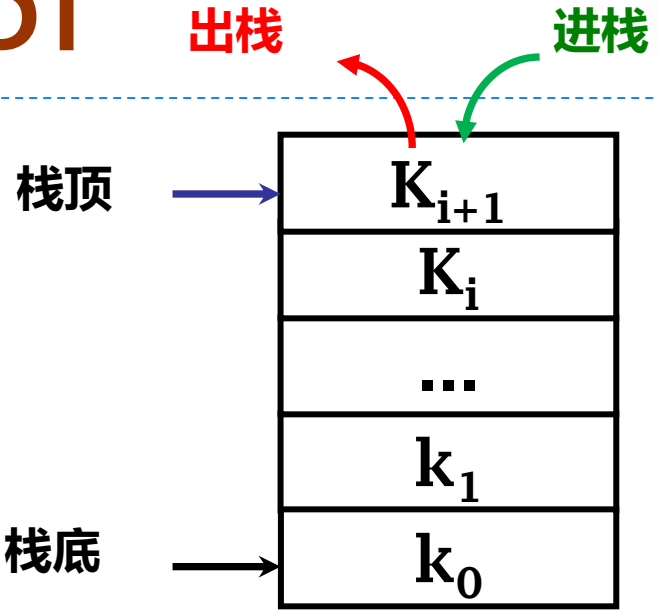
抽象数据类型ADT

- 抽象数据结构三元组

  <数据对象D，数据关系S，数据操作P>

- 先定义逻辑结构，再定义运算
  - **逻辑结构**：数据对象 + 数据关系
  - **运算**：数据操作

例：栈的抽象数据类型ADT



- 逻辑结构：线性表
- 操作特点：限制访问端口
  - 只允许在一端进行插入、删除操作
  - 入栈（push）、出栈（pop）、取栈顶（top）
    判栈空（isEmpty）

```
template <class T>                  // 栈的元素类型为 T
class Stack {
public:                             // 栈的运算集
    void clear();                   // 变为空栈
    bool push(const T item);        // item入栈，成功返回真，否则假
    bool pop(T & item);             // 弹栈顶，成功返回真，否则返回假
    bool top(T& item);              // 读栈顶但不弹出，成功真，否则假
    bool isEmpty(;                  // 若栈已空返回真
    bool isFull();                  // 若栈已满返回真
};
```

第1章　概论

- 问题求解
- 数据结构及抽象数据类型
- **算法的特性及分类**
- **算法的效率度量**

问题 —— 算法 —— 程序

目标：问题求解

- **问题（problem）**一个函数
  - 从输入到输出的一种映射
- **算法（algorithm）**一种方法
  - 对特定问题求解过程的描述，是指令的有限序列
- **程序（program）**
  - 是算法在计算机程序设计语言中的实现



算法的特性

- 通用性
  - 对参数化输入进行问题求解
  - 保证计算结果的正确性
- 有效性
  - 算法是有限条指令组成的指令序列
  - 即由一系列具体步骤组成
- 确定性
  - 算法描述中的下一步应执行的步骤必须明确
- 有穷性
  - 算法的执行必须在有限步内结束
  - 换句话说，算法不能含有死循环

算法分析

- 算法的时间代价(时间复杂性)：
  - 当问题规模以某种单位由1增至n时，对应算法所耗费的时间也以某种单位由F(1)增至F(n)，这时我们称该算法的时间代价是F(n)。
  - 时间单位：一般规定为执行一个简单语句（如赋值语句、判断语句等）所用时间。
- 算法的空间代价(空间复杂性)：
  - 当问题规模以某种单位由1增至n时，解该问题的算法所需占用的空间也以某种单位由G(1)增至G(n)，这时我们称该算法的空间代价是G(n)。
  - 空间单位：一般规定为一个简单变量（如整型、实型等）所占存储空间的大小。

算法的渐进分析

$$f(n) = n^2 + 100n + \log_{10} n + 1000$$

- 数据规模 n 逐步增大时，

  f(n) 的增长趋势

- 当 n 增大到一定值以后，计算公式中影响最大的就是 n 的幂次最高的项
  - 其他的常数项和低幂次项都可以忽略

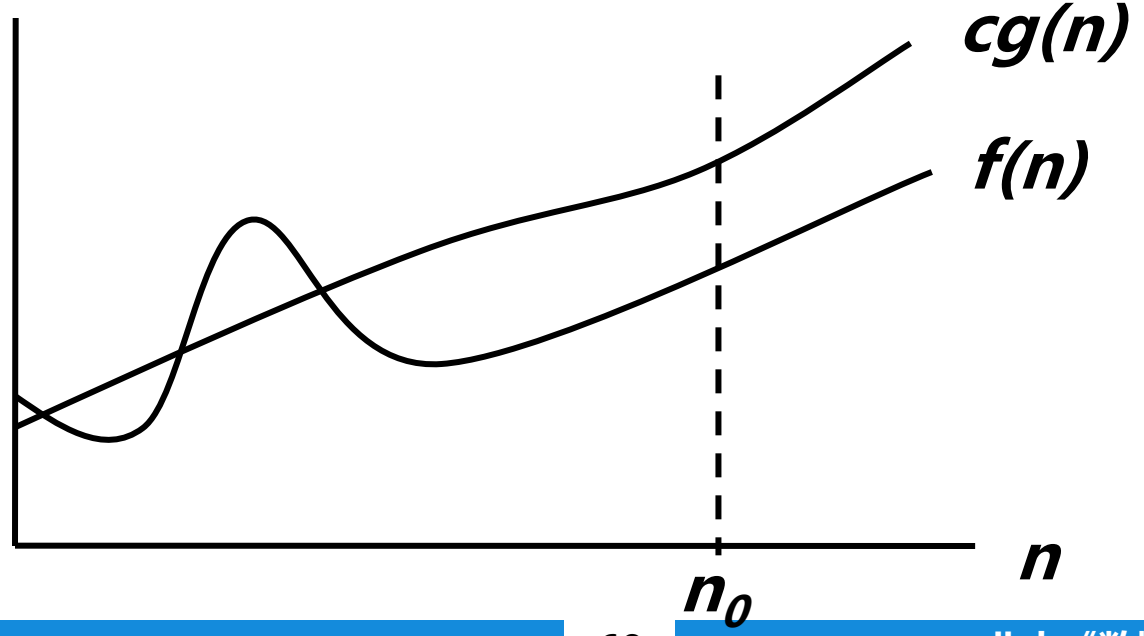
算法渐进分析：大O表式法

- 函数f，g定义域为自然数，值域非负实数集
- **定义**：如果存在正数c和n_0，使得对任意的$n \geq n_0$， 都有 $f(n) \leq cg(n)$，
- 称f(n)在集合O(g(n))中，简称f(n)是O(g(n))的，
  或 f(n) = O(g(n))
- 大O表示法：表达函数增长率上限
  - 一个函数增长率的上限可能不止一个
  - 采用大O表示法时，最好给出所有上限中**最“紧”**（也即最小）的那个上限

若$f(n)=3n^3$，c=3，$g(n)=n^3$ ，则$f(n)=O(n^3)$。

大 O 表示法

- f(n) = O(g(n))，当且仅当
  — 存在两个参数 c > 0 ， n_0 > 0，对于所有的 n ≥ n_0 ， 都有 f(n) ≤ cg(n)
- iff ∃ c, n_0 > 0 s.t. ∀ n ≥ n_0 : 0 ≤ f(n) ≤ cg(n)



n足够大
g(n)是 f(n) 的上界

大O表示法的单位时间

- 简单布尔或算术运算
- 简单I/O
  - 指函数的输入/输出
    例如，从数组读数据等操作
  - 不包括键盘文件等I/O
- 函数返回

大O表示法的运算法则

- **加法规则:** $f_1(n)+f_2(n)=O(\max(f_1(n), f_2(n)))$
  - 顺序结构，if 结构，switch结构
- **乘法规则:** $f_1(n)\cdot f_2(n)=O(f_1(n)\cdot f_2(n))$
  - for, while, do-while结构

```
for (i=0; i<n; i++)
    for (j=i; j<n; j++)
        k++;
```

$n-i$

?

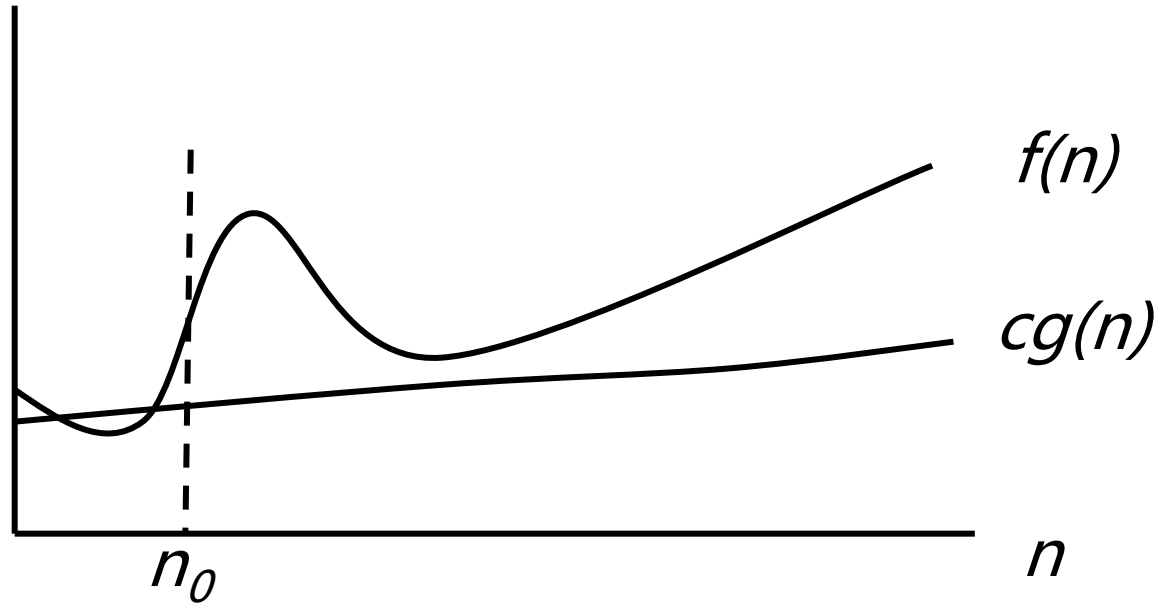
$$\sum_{i=0}^{n-1}(n-i)=\frac{n(n-1)}{2}=\frac{n^2-n}{2}=O(n^2)$$

算法渐进分析： Ω表式法

- **定义**：如果存在正数c和n_0，使得对所有的$n \geq n_0$，都有 $f(n) \geq cg(n)$，则称f(n)在集合Ω (g(n)) 中，或简称 f(n) 是 Ω (g(n))的， 或 f(n) = Ω (g(n))
- 大O表示法和大Ω表示法的唯一区别在于不等式的方向而已
- 采用大Ω表示法时， 最好找出在函数增值率的所有下限中那个最“紧”（即最大）的下限

大 Ω 表示法

- f(n) = Ω(g(n))
  - iff ∃ **c, n_0 > 0 s.t. ∀ n ≥ n0 , 0 ≤ cg(n) ≤ f(n)**
- 与大O表示法的唯一区别在于不等式的方向



n足够大
g(n)是 f(n) 的下界

算法渐进分析： Θ表式法

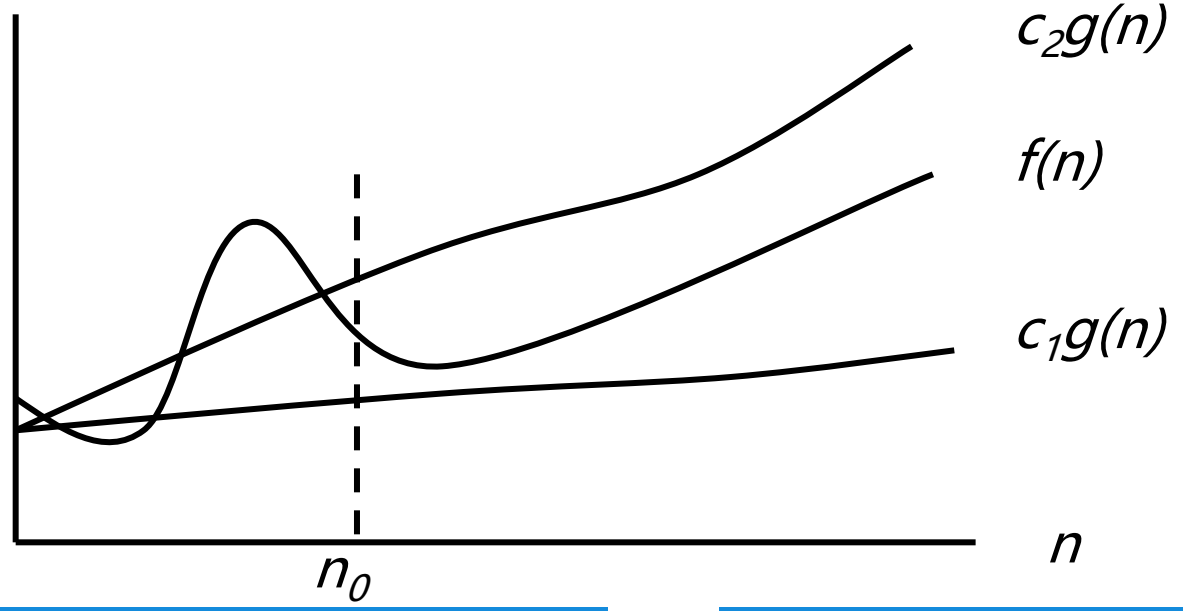
- 当上、下限相同时则可用 Θ 表示法
- 定义如下：

  如果一个函数既在集合O (g(n))中又在集合Ω (g(n))中，则称其为Θ (g(n))。
- 也即，当上、下限相同时则可用 Θ 表示法
- 存在正常数c_1, c_2，以及正整数n_0，使得对于任意的正整数$n > n_0$，有下列两不等式同时成立：

$$c_1\, g(n) \le f(n) \le c_2\, g(n)$$

大 Θ 表示法

- f(n) = Θ(g(n))
  - iff ∃ $c_1, c_2, n_0 > 0$ s.t. $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$, ∀ $n >= n_0$
- 上、下限相同，则可用 Θ 表示法



n足够大
f(n) 与 g(n) 增长率一样

1.4 算法复杂性分析

考虑如下程序：

```
{
    action1(....);
    action2(....);
}
```

其中action1的时间代价是T1(n) = O(n^2)，
action2的时间代价是T2(n) = $O(n^3)$。
按照加法规则，计算整个程序的时间代价为：

$T(n) = T1(n) + T2(n) = O(\max(n^2, n^3)) = O(n^3)$

(1) i=1; k=0;
while(i<n)
{ k=k+10*i;i++; }

◆ T(n)=n-1
∴ T(n)=O(n)
◇ 这个函数是按线性阶递增的

(2) x=91; y=100;
while(y>0)
if (x>100)
{x=x-10; y--;}
else x++;

◆ T(n)=O(1)
◇ 这个程序看起来有点吓人，
while循环总共运行了1100次，但是我们看到这段程序的运行是和n无关的，只是一个常数阶的函数。

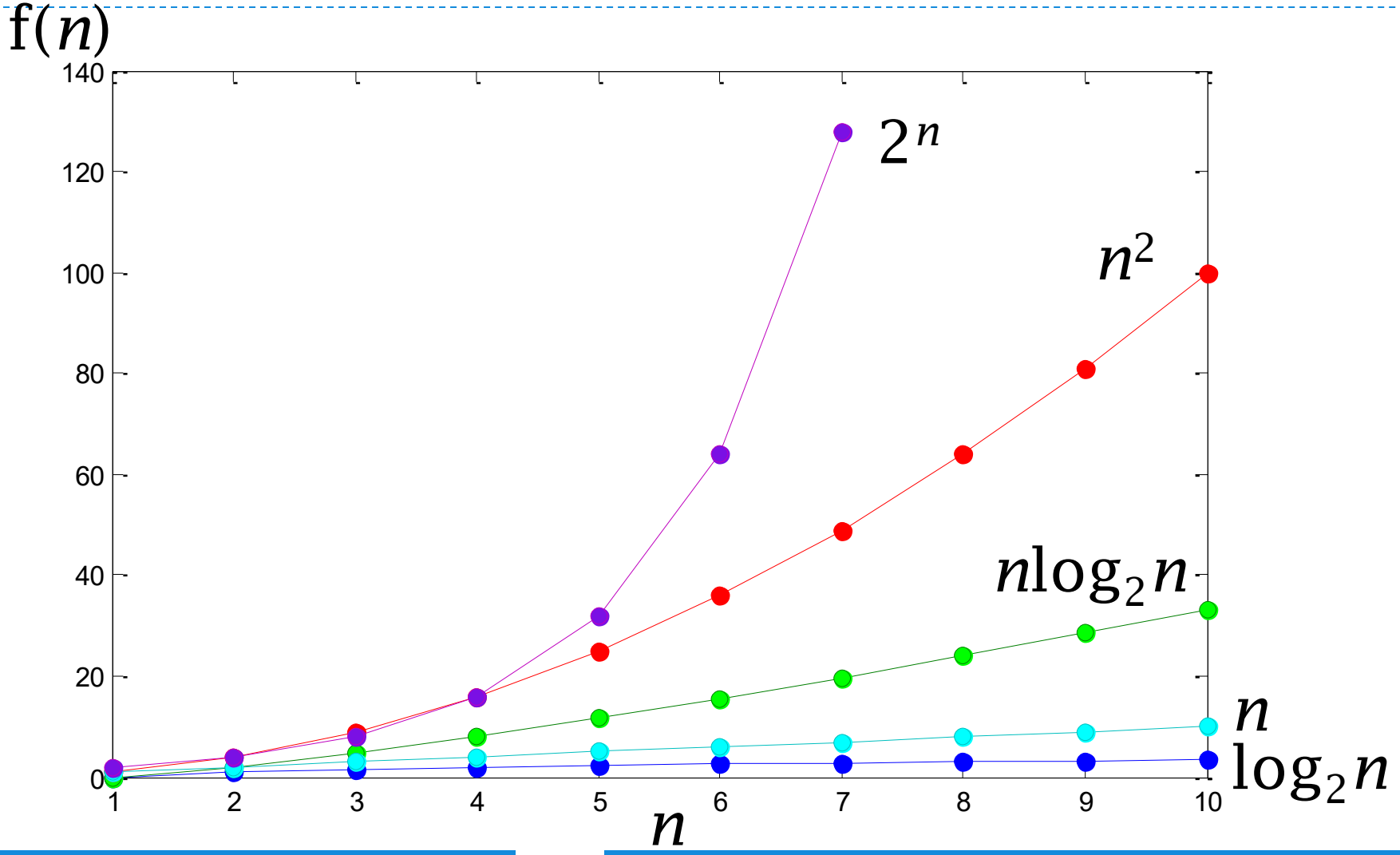
(3) i=1; j=0;
while(i+j<=n)
{j++; i++;}

◆ T(n)=n/2
∴ T(n)=O(n)
◇ 虽然时间函数是n/2,但其数量级仍是按线性阶递增的。

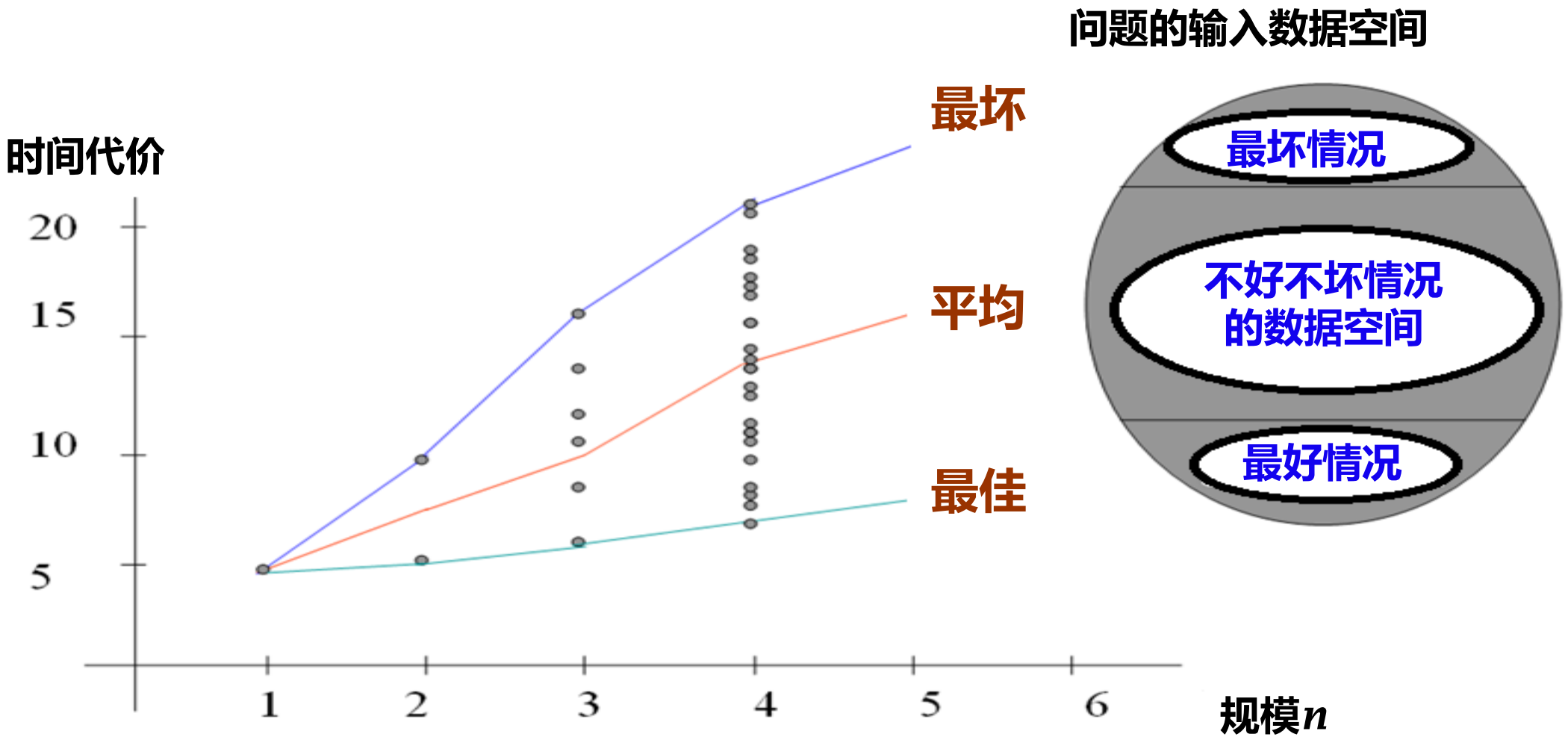
(4) x=n; // n>1
while (x>=2^(y+1)) // 伪代码，$2^{(y+1)}$
y++;

◆ $T(n)=\log_2 n$
∴ $T(n)=O(\log_2 n)$
◇ 最坏的情况是y=0，
那么循环的次数是$\log_2 x$次，
也就是$\log_2 n$次(n>1)。
这是一个按对数阶递增的函数。

增长率函数曲线



问题空间 vs 时间开销


时间代价
20
15
10
5
1
2
3
4
5
6
规模n
最坏
平均
最佳

问题的输入数据空间
最坏情况
不好不坏情况
的数据空间
最好情况

执行次数随输入数据不同而不同

例：冒泡排序法

```
void Bubble_Sort(int a[], int n)
{       int i, change=TRUE;
        for(i=n-1; i >1 && change; --i)
        {       change = FALSE;
                for(j = 0; j < i; j++)
                        if(a[j] > a[j+1])
                        {       swap(&a[j], &a[j+1]);
                                change = TRUE;
                        }
        }
}
```

a = {1,2,3,4,5,6,7,8,9}
a = {9,8,7,6,5,4,3,2,1}



顺序找k值

- 顺序从一个规模为n的一维数组中找出一个给定的K值
- 最佳情况
  - 数组中第1个元素就是K
  - 只要检查一个元素
- 最差情况
  - K是数组的最后一个元素
  - 检查数组中所有的n个元素

顺序找k值——平均情况

- 如果等概率分布
- K值出现在n个位置上概率都是1/n则平均代价

$$\frac{1+2+...+n}{n} = \frac{n+1}{2}$$

顺序找k值——平均情况

- 概率不等
  - 出现在第1个位置的概率为1/2
  - 第2个位置上的概率为1/4
  - 出现在其他位置的概率都是

$$\frac{1-1/2-1/4}{n-2}=\frac{1}{4(n-2)}$$

  - 平均代价

$$\frac{1}{2}+\frac{2}{4}+\frac{3+\ldots+n}{4(n-2)}=1+\frac{n(n+1)-6}{8(n-2)}=1+\frac{n+3}{8}$$

二分法检索

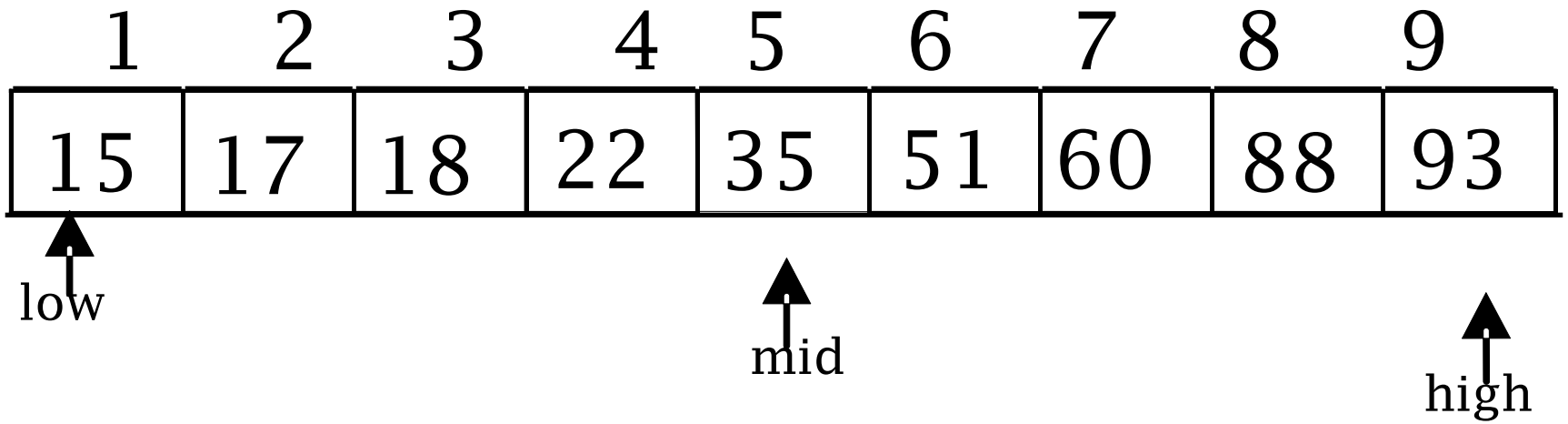
对于已排序顺序线性表

- 数组中间位置的元素值 k_{mid}
  - 如果$k_{mid} = k$，那么检索工作就完成了
  - 当$k_{mid} > k$时，检索继续在前半部分进行
  - 相反地，若$k_{mid} < k$，就可以忽略mid以前的那部分，检索继续在后半部分进行
- 快速
  - $k_{mid} = k$ 结束
  - $K_{mid} \neq k$ 起码缩小了一半的检索范围

二分法检索算法

```
template  <class Type> int BinSearch (vector<Item<Type>*>& dataList, int length, Type k){
    int low=1, high=length, mid;
    while (low<=high)  {
        mid=(low+high)/2;
        if (k<dataList[mid]->getKey())
                high = mid-1;                  //右缩检索区间
        else if (k>dataList[mid]->getKey())
                low = mid+1;                   //左缩检索区间
        else return mid;                       //成功返回位置
     }
    return 0; //检索失败，返回0
}
```

二分法检索图示

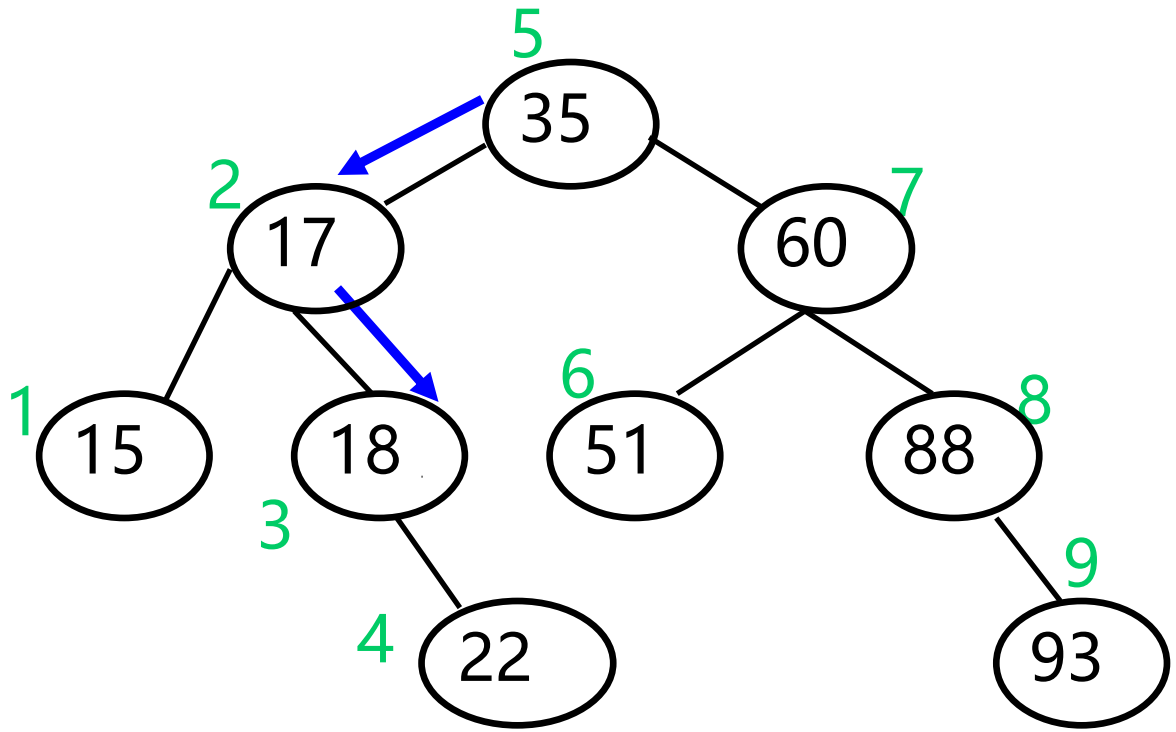


检索关键码18 low=1 high=9 K=18

第一次：mid=5; array[5]=35>18
high=4; (low=1)
第二次：mid=2; array[2]=17<18
low=3; (high=4)
第三次：mid=3; array[3]=18=18
mid=3； return 3

二分法检索性能分析

- 最大检索长度为
  $\lceil \log_2(n+1) \rceil$
- 失败的检索长度是
  $\lceil \log_2(n+1) \rceil$
  或 $\lfloor \log_2(n+1) \rfloor$
- 在算法复杂性分析中
  - log n是以2为底的对数
  - 以其他数值为底，算法量级不变



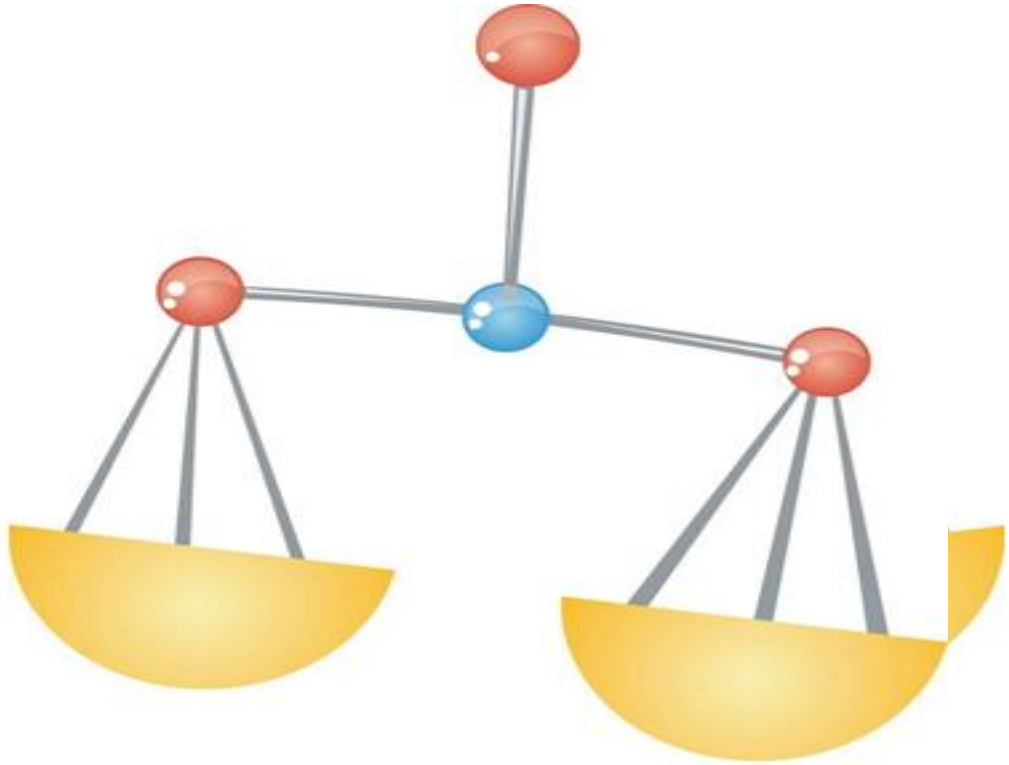


时间/空间权衡

- **数据结构**
  - 一定的空间来存储它的每一个数据项
  - 一定的时间来执行单个基本操作
- **代价和效益**
  - 空间和时间的限制
  - 软件工程

时空权衡

- 增大空间开销可能改善算法的时间开销
- 可以节省空间，往往需要增大运算时间



数据结构和算法的选择

- 仔细分析所要解决的问题
  - 特别是求解问题所涉及的数据类型和数据间逻辑关系
    ——— 问题抽象、数据抽象
  - 数据结构的初步设计往往先于算法设计
- 注意数据结构的可扩展性
  - 考虑当输入数据的规模发生改变时，
    数据结构是否能够适应求解问题的演变和扩展

数据结构与算法



- 教材：**《数据结构与算法》**。张铭，王腾蛟，赵海燕 编，高等教育出版社，2008年 6月。——普通高等教育“十一五”国家级规划教材（入选“十二五”）
- 国家精品课网站 http://jpk.pku.edu.cn/course/sjjg/

练习

- 按照增值率从低到高的顺序排列以下表达式：
  - $4n^2$, $\log_3 n$, ${}_3n$, ${}_{20}n$, 2000, $\log_2 n$, $n^{2/3}$
  - 并给n!在其中找个适当的位置

斯特林公式

$$\lim_{n\to+\infty}\frac{n!}{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n}=1$$